

Lab Manual

DSC293 Data Science

Fundamentals



CUI

Lab Contents:

This course will introduce concepts, techniques and tools for Data Science. Topics include: Data Science Foundation; Data Science Process; Introduction to R & RStudio; Data Wrangling; Data Visualization; Statistical Inference; Predictive Modeling; and Data Science Ethics.

Student Outcomes (SO)

S.#	Description
2	Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines
3	Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations
4	Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations
5	Function effectively as an individual and as a member or leader in diverse teams and in multi-disciplinary settings.

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -1	Develop a project based on data science concepts in a team environment.	Creating	2-5

Lab Assessment Policy

The lab work done by the student is evaluated using Psycho-motor rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

Lab #	Main Topic	Page #
Lab 01	Installing RStudio, understanding IDE, how to create a project, Basics of R (creating objects, assignment, addition, subtraction etc), how wo create R script	4
Lab 02	Basic control structures if, if-else, loops	6
Lab 03	Installing packages, loading data, scatter plot, histograms,	13
Lab 04	Project 01: Sentiment Data Analysis	23
Lab 05	Project 01: Sentiment Data Analysis	29
Lab 06	Project 01: Sentiment Data Analysis	34
Lab 07	Project 02: Uber Data Analysis	40
Lab 08	Mid Term Exam	
Lab 09	Project 02: Uber Data Analysis	48
Lab 10	Project 03: Movie Recommendation	56
Lab 11	Project 03: Movie Recommendation	64
Lab 12	Project 03: Movie Recommendation	91
Lab 13	Project 04: Credit Card Fraud Detection	77
Lab 14	Project 04: Credit Card Fraud Detection	88
Lab 15	Final Term Exam	

Lab 01

Installing RStudio

Objective:

The objective of this lab is to setup RStudio on your PC/Laptop.

Activity Outcomes:

The activities provide guidance to the following

- Downloading R Studio.
- Understanding RStudio IDE.
- Create a new project, setup working director, understanding the environment variables used.

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your RStudio

Useful Concepts

Downloading and Installing RStudio: please visit the following link: <https://www.rstudio.com/products/rstudio/download/> OR you can download anaconda that includes RStudio as well from the link: <https://www.anaconda.com/products/individual>. After downloading the installer of your choice, double click the file and follow the instructions to install the RStudio on your PC/Laptop.

Objects: R stores information and operates on **Objects**. Simplest form is scalar, vectors and matrices. Complex objects include data frames and lists.

Arithmetic Operations: R performs basic operations on console or via R script (explained in next lab). We can use simple statements like $4 + 6$ to calculate the expression and see the result immediately on console

Assignment Operator: Assignment is performed using the `<-` sign. For example if we are to assign the value of 6 to x then we write `x<-6`.

Concatenation Operator: if we want to create a list or array of elements then we use c operator. E.g. if we want to assign a list of 3 values i.e 2,4,6 to z then we write `z<-c(2,4,6)`

Colon Operator: colon operator is used for assigning values in a range. E.g. if we want to assign values from 1 to 10 to x, then we write `x<-1:10`

seq Operator: if we want to generate a sequence then we use seq function of R. There are few optional but useful arguments to this function e.g. *length*: that generates that many numbers with equal difference. *by*: defines the step size. E.g. `seq(1,10)` will generate sequence from 1 to 10 with default step size of 1. `Seq(1,10, by=2)` will generate sequence of 1 to 10 with step size of 2 i.e 1 3 5 7 9

rep function: rep is short for repetition. `rep(0,100)` will generate 0, 100 times. `Rep (1:3,6)` will generate number 1,2,3 and rep this sequence times

Matrices: matrices can be built using `cbind` (for column bind) and `rbind` (for row bind). For

example, if `x<-c(5,7,9)` and `y<-c(6,3,4)` then we can create a matrix `z<-cbind(x,y)`. we can also bind the `z` row-wise using `rbind` as `a<-rbind(z,z)`. you can further explore the additional options of `nrow` and `byrow` etc for making yourselves more familiar with how to create matrices in R

Loading Datasets: to load data sets R uses the `data(<dataset name>)` function. Suppose we want to load the data set of *trees* then we write `data(tress)` to load this data set

Help: to get help in R about any function we write `help(<function name>)` for example if we want to see the help on how to create a normal distribution (*dnorm*), we can write `help(dnorm)`

Solved Lab Activites

Activity 1:

R program to illustrate while loop

Solution:

Lab 02

Basic Control Structures

Objective:

The objective of this lab is to understand basic control structures in R.

Activity Outcomes:

The activities provide guidance to the following

- Understanding and implementation of if-else.
- Understanding and implementation of for and while loops in R.

Instructor Note:

You need to have a working setup of RStudio on the Laptop/PC for writing sample R codes to test conditional and looping structures.

1) Useful Concepts:

Flowchart:

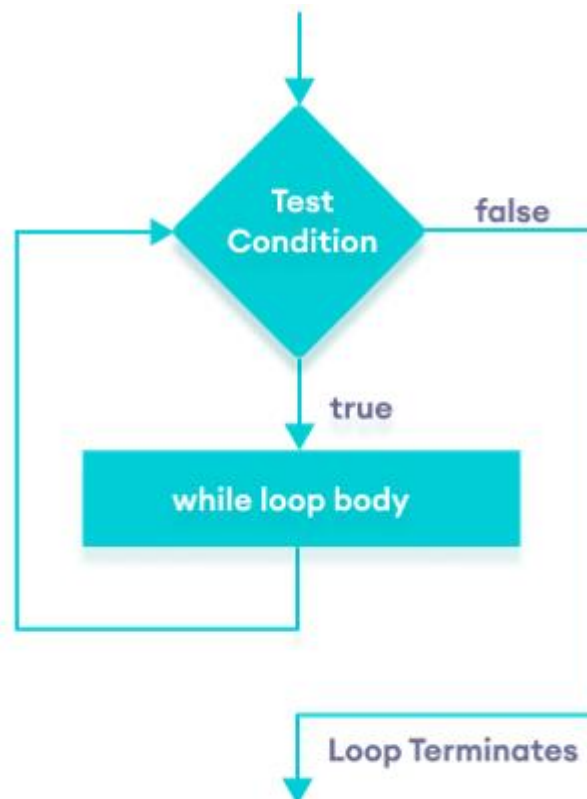


Figure 1:Flowchart of while loop

Syntax :

```
while (boolean condition){  
  loop statements...  
}
```

Looping in programming languages is a feature which facilitates the execution of a set of instructions/functions repeatedly while some condition evaluates to true. R provides two ways for executing the loops. While all the ways provide similar basic functionality, they differ in their syntax and condition checking time.

1. **Taking input:** In R we take input using the following command:
Num <- readline(prompt("Enter Number"))
2. **while loop:** A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.
3. **for loop:** for loop provides a concise way of writing the loop structure. Unlike a while loop, a for statement consumes the initialization, condition and increment/decrement in one line thereby providing a shorter, easy to debug structure of looping.

Flowchart:

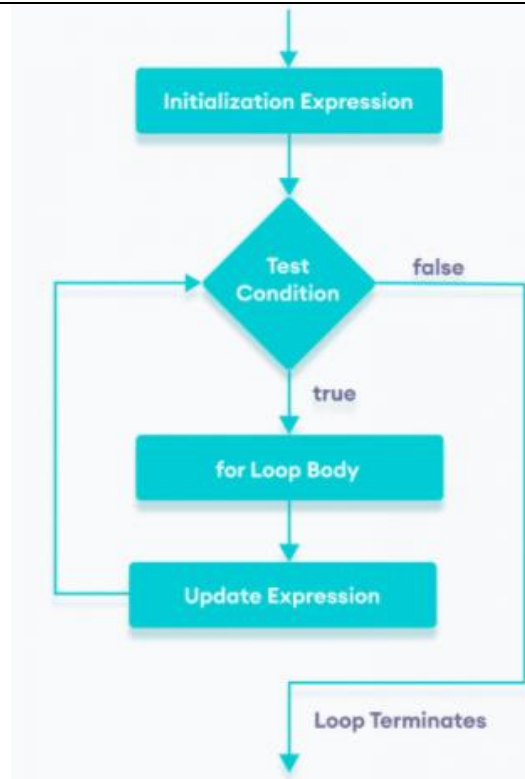


Figure 2:Flowchart of for loop

Syntax :

```
for (condition){  
    statement(s)  
}
```

4. **if statement:** if structure checks on a condition, and if the condition is true it executes the associated code otherwise R skips the associated code and jumps to the next level

Flowchart:

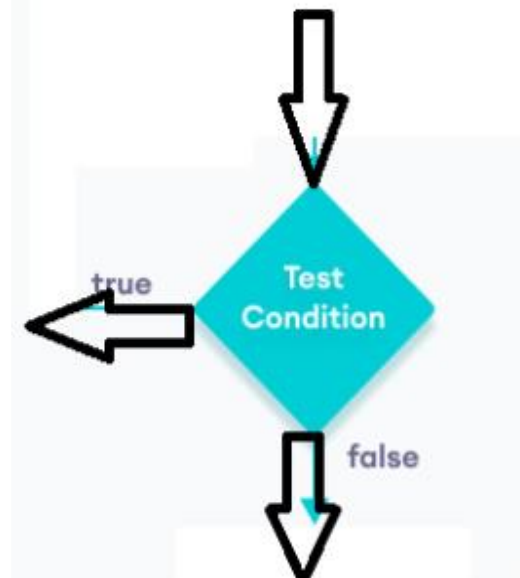


Figure 3:Flowchart of if statement

Syntax :

```
if (condition) {  
    statements..  
}
```

5. **if-else statement:** In R, if-else structure checks on a condition, and if the condition is true it executes the associated code otherwise the code block of the else-block is executed

Flowchart:

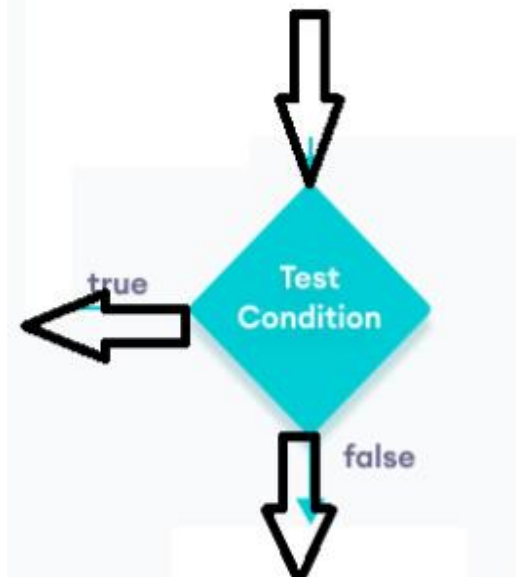


Figure 4:Flowchart of if-else

Syntax :

```
if (condition) {  
    statements..  
} else if (condition) {  
    statements..  
} else {  
    statements..  
}
```

2) Solved Lab Activities (Allocated Time 1 Hr. The time can vary)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Print greater of the two numbers

Solution:

```
a <- 33  
b <- 200
```

```
if (b > a) {  
  print("b is greater than a")  
}
```

Output

b is greater than a

Activity 2:

R program to illustrate if-else

```
a <- 33  
b <- 33  
  
if (b > a) {  
  print("b is greater than a")  
} else if (a == b) {  
  print("a and b are equal")  
}
```

Output

a and b are equal

Activity 3:

R program to illustrate nested if

```
x <- 41  
  
if (x > 10) {  
  print("Above ten")  
  if (x > 20) {  
    print("and also above 20!")  
  } else {  
    print("but not above 20.")  
  }  
} else {  
  print("below 10.")  
}
```

Output

Above ten and also above 20!

Activity 4:

R program to illustrate while loop

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
}
```

Output

1
2
3
4
5

Activity 5:

R program to illustrate while loop with skip

```
i <- 0
while (i < 6) {
  i <- i + 1
  if (i == 3) {
    next
  }
  print(i)
}
```

Output

1
2
4
5
6

Activity 6:

R program to illustrate while loop with break

```
i <- 1
```

```
while (i < 6) {  
  print(i)  
  i <- i + 1  
  if (i == 4) {  
    break  
  }  
}
```

Output

1
2
3
4

Activity 7:

R program to illustrate for loop

```
for (x in 1:5) {  
  print(x)  
}
```

Output

1
2
3
4
5

Activity 8:

R program to illustrate for loop on a list

```
fruits <- list("apple", "banana", "cherry")  
  
for (x in fruits) {  
  print(x)  
}
```

Output

apple
banana
cherry

Activity 9:

R program to illustrate nested for loop on lists

```
adj <- list("red", "big", "tasty")  
  
fruits <- list("apple", "banana", "cherry")  
  for (x in adj) {  
    for (y in fruits) {  
      print(paste(x, y))  
    }  
  }
```

Output

red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry

3) Graded Lab Tasks (Allocated Time 1 Hr. **The time can vary**)

(The instructor can design the complexity and frequency of unsolved lab tasks as per requirement of the lab. The lab tasks assigned by the instructor should be evaluated in the same lab.)

Lab Task 1

Write a program that prompts the user to input an integer and then outputs STAR OF DAVID with maximum number of stars in the maximum line as to what the user has given input of

Lab 03

Installing packages, loading data, basic data visualization

Objective:

The objective of this lab is to understand basic control structures in R.

Activity Outcomes:

The activities provide guidance to the following

- Understanding how to implement packages in R
- Understanding how to load data in R
- Understanding how to do basic data visualization using histograms and scatter plots,

Instructor Note:

You need to have a working setup of RStudio on the Laptop/PC for writing sample R codes to test different packages, load sample data sets and perform basic visualization techniques.

4) Useful Concepts

Packages are part of R programming and they are useful in collecting sets of R functions into a single unit. It also contains compiled code and sample data. All of these are kept stored in a directory called the "library" in the R environment

1. **Packages and library:** R packages are a collection of R functions, complied code and sample data. They are stored under a directory called "library" in the R environment. By default, R installs a set of packages during installation. More packages are added later, when they are needed for some specific purpose
2. **installed.packages():** is the command to check which packages are installed in R
3. **install.packages("package__name"):** is the command to install a package in R. e.g. if we want to install package "tidytext" we will write `install.package("tidytext")`
4. **library("dataset"):** is the command to load data set in R e.g. if we want to load "iris" dataset in R we will use the simple command of `library("datasets")`
5. **head(dataset_name):** in R head function returns the first records of the selected dataset e.g. `head(iris)` will show the first records of the "iris data set loaded via command `library("datasets")`".
6. **summary(dataset_name[\$attribute]):** in R `summary()` is a generic function that returns the summary of the selected dataset or the selected variable e.g. `summary(iris)` will show the first records of the "iris data set loaded via command `library("datasets")`".
7. **summary(dataset_name[\$attribute]):** in R `summary` function returns the summary of the selected dataset or the selected variable e.g. `summary(iris)` will show the summary of the "iris data set loaded via command `library("datasets")`" and `summary(iris$Sepal.Length)` will show the summary of the "iris data set loaded via command `library("datasets")`"
8. **plot(dataset_name[\$attribute]):** in R, `plot()` function created the graph of the selected dataset or the selected variable e.g. `plot(iris)` will create graphs of the "iris data set loaded via command `library("datasets")`" and `plot(iris$Species)` will create graph of the "iris data set loaded via command `library("datasets")`"
9. **plot(dataset_name[\$attribute]):** in R, `plot()` function created the graph of the selected dataset or the selected variable e.g. `plot(iris)` will create graphs of the "iris data set loaded via command `library("datasets")`" and `plot(iris$Species)` will create graph of the "iris data set loaded via command

```
library("datasets")
```

5) Solved Lab Activities (Allocated Time 1 Hr. **The time can vary**)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

R program to install packages

```
install.packages("tidytext")  
install.packages("ggplot2")
```

Output

The required packages will be installed

Activity 2:

R program to include libraries

```
library(tidytext)  
library(ggplot2)
```

Output

The required libraries will be included/imported

Activity 3:

R program to load datasets (built-in) and view header

```
library(datasets)  
  
#Loading iris datasets from the library and displaying the header  
head(iris)
```

Output

```
Sepal.Length Sepal.Width Petal.Length Petal.Width Species
```


1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Activity 4:

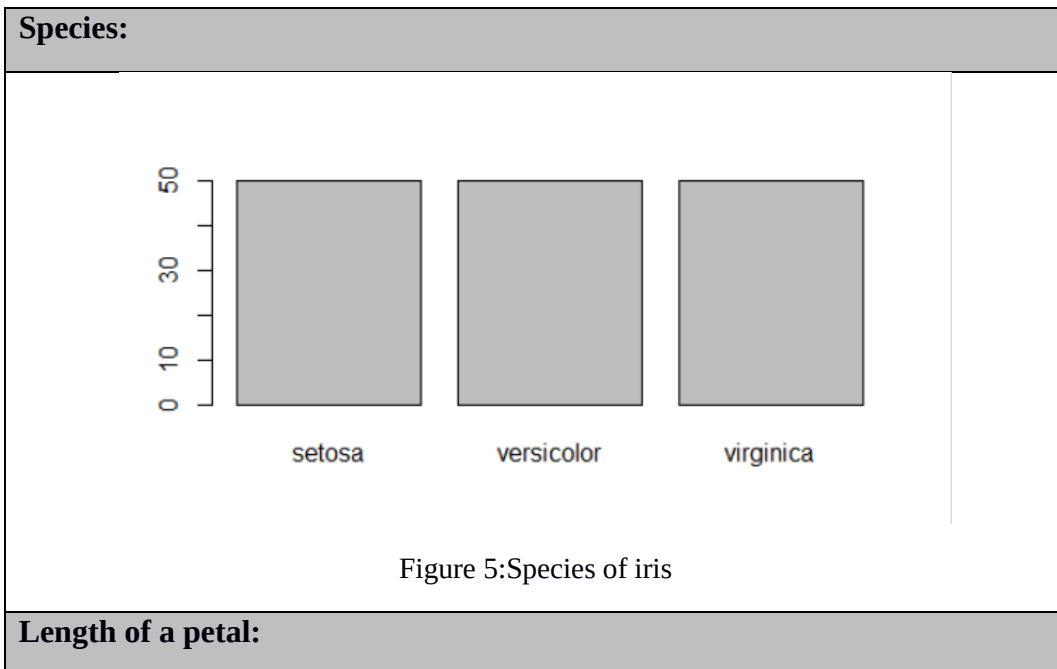
R program to plot datasets

```
library(datasets)

#Loading iris datasets from the library and displaying the header
head(iris)

#usint plot function to plot different graphs
plot(iris$Species) # Categorical variable
plot(iris$Petal.Length) # Quantitative variable
plot(iris$Species, iris$Petal.Width) # Cat x quant
plot(iris$Petal.Length, iris$Petal.Width) # Quant pair
plot(iris) # Entire data frame
```

Output:



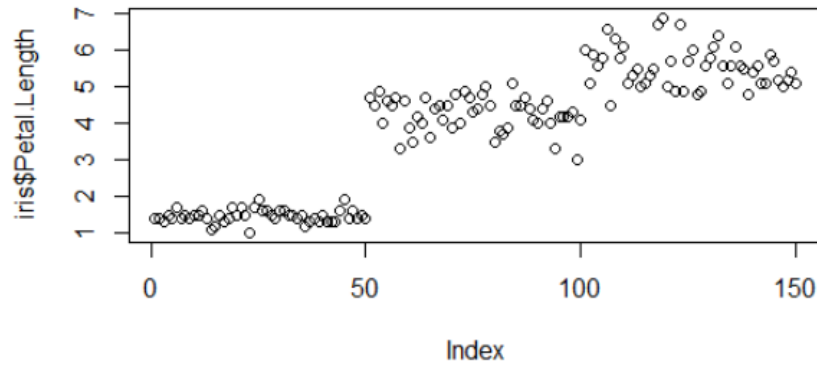


Figure 6 :Length of petal

Species and width of a petal:

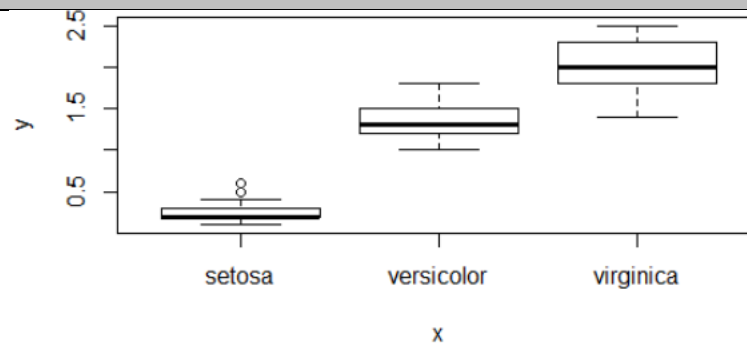


Figure 6 :Species and width of petal

Length and width of a petal:

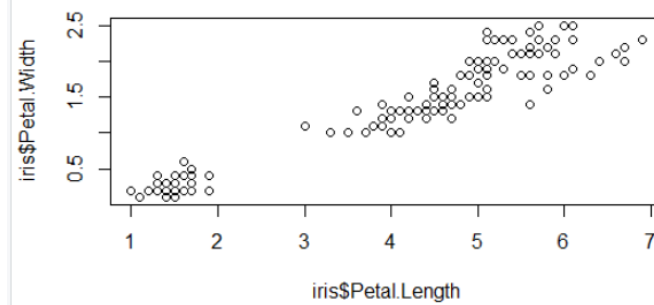
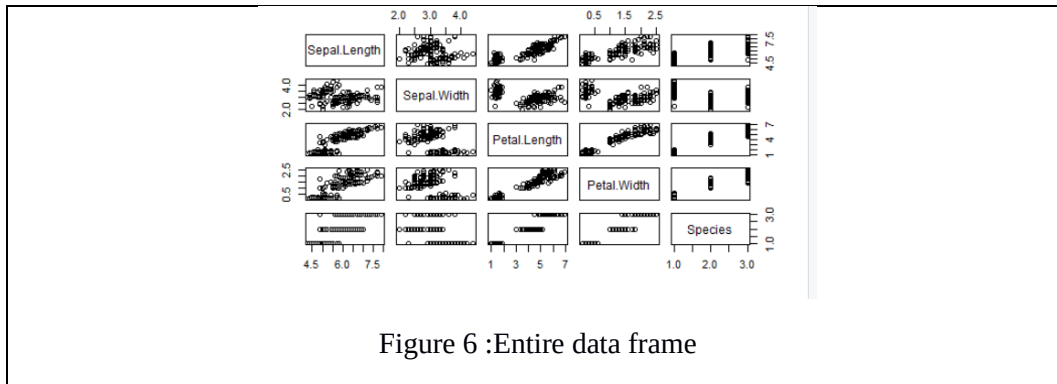


Figure 7 :Length and width of petal

Entire Data Frame

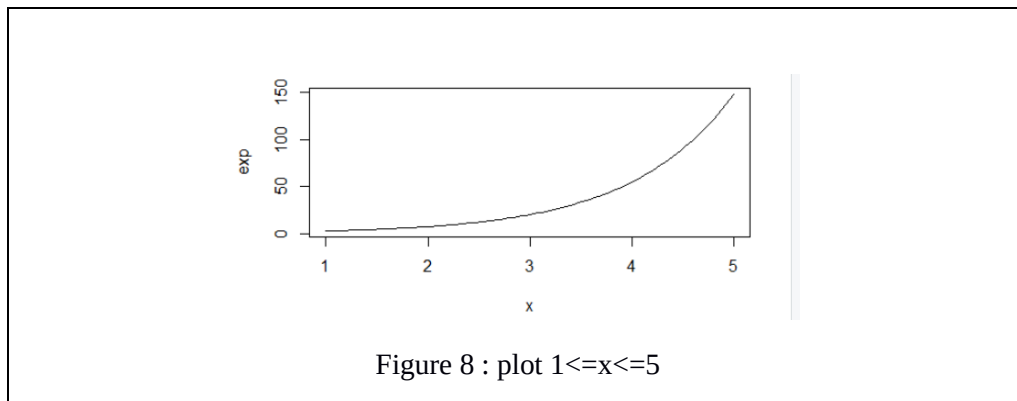


6) Graded Lab Tasks((Allocated Time 1 Hr. **The time can vary**)
(The instructor can design the complexity and frequency of unsolved lab tasks as per requirement of the lab. The lab tasks assigned by the instructor should be evaluated in the same lab.)

Lab Task 1:

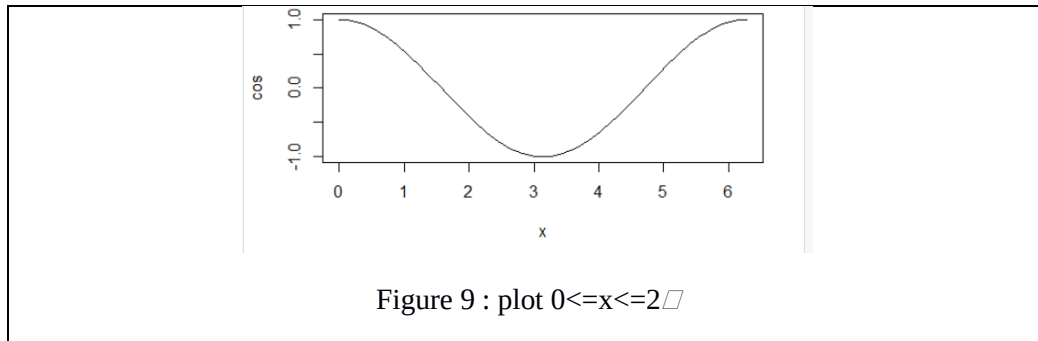
1. Plot e^x in R for $1 \leq x \leq 5$

Output:



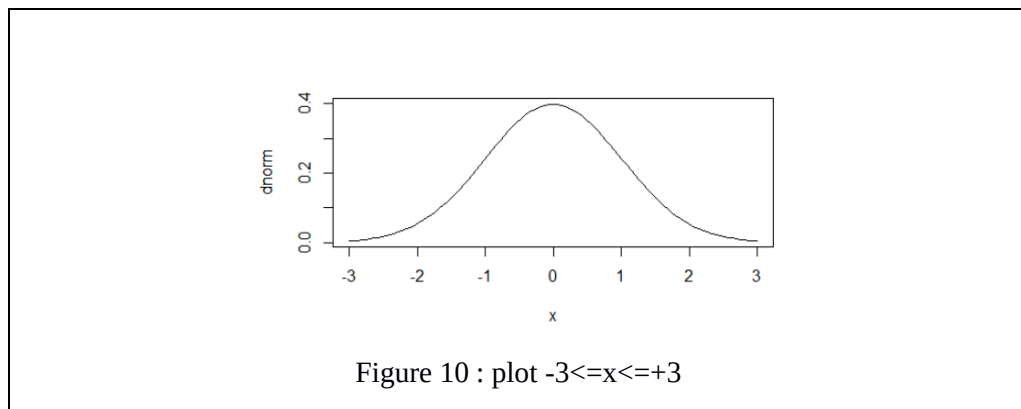
2. Plot $\cos(x)$ in R for $0 \leq x \leq 2\pi$

Output:



3. Plot normal distribution over x in R for $-3 \leq x \leq +3$

Output:

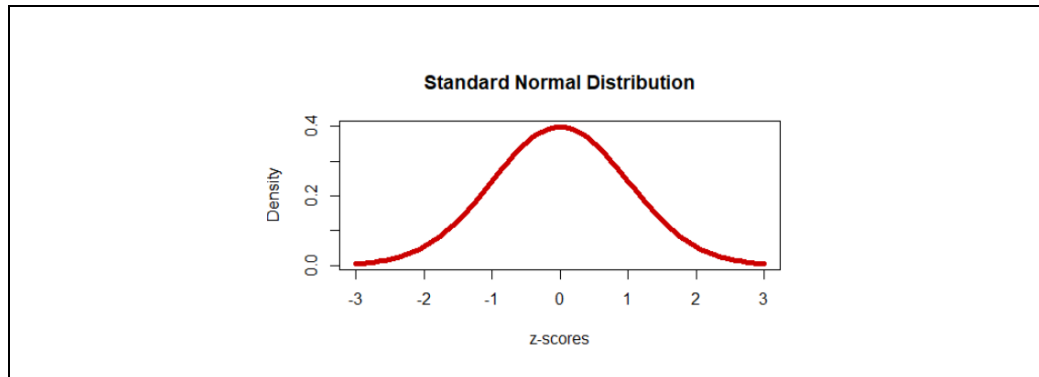


Working Code:

```
plot(cos, 0, 2*pi)
plot(exp, 1, 5)
plot(dnorm, -3, +3)
```

Lab Task 2:

Use the plot function to display graph of normal distribution from -3 to +3 with line width of units 5, and line color to be red, along with label on the main chart and the x and y axis.

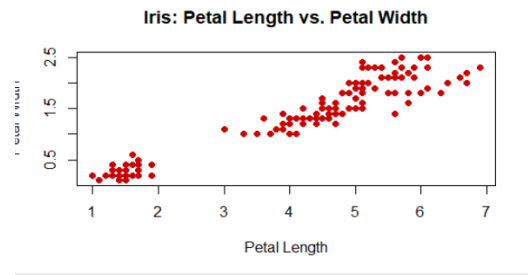


Working Code:

```
plot(dnorm, -3, +3,  
     col = "#cc0000",  
     lwd = 5,  
     main = "Standard Normal Distribution",  
     xlab = "z-scores",  
     ylab = "Density")
```

Lab Task 3:

Use the plot function with few options like changing the color to red, creating solid points, labeling the main chart as well as the x-axis and the y-axis



Working Code:

```
plot(iris$Petal.Length, iris$Petal.Width,  
     col = "#cc0000", # Hex code for datalab.cc red  
     pch = 19,        # Use solid circles for points  
     main = "Iris: Petal Length vs. Petal Width",  
     xlab = "Petal Length",  
     ylab = "Petal Width")
```

LAB 04

Project 1

Sentimental Analysis

Objective:

The objective of this lab is to get a basic understanding of how a model for sentimental analysis works.

Activity Outcomes:

The activities provide guidance to the following

- Understanding what sentiments are
- Learn to perform basic data wrangling processes on sentimental analysis
- Learning about the basic procedures involved in performing tidy operations on a dataset

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your Rstudio

7) Useful Concepts

Sentiments can be referred to as a feeling or attitude that are derived through our feelings. While sentiments are mostly used as a psychological term, the purpose of this sentimental model is to help evaluate positive/negative sentiments through words which are loaded from a Janeaustenr dataset.

1. **Lexicon:** Lexicons are a common term used in sentiment analysis which are used to describe and calculate how sentiments are evaluated from textual data. There are three main categories of lexicons which are: AFFIN, Bing, Loughran
2. **%>% or Forward Pipe Line Operator:** A forward pipe line operator in R is used to chain multiple set of commands. As the expressions are evaluated, the value keeps getting passed on as long as the operator is being used between statements, resulting in the output of the whole expression.
3. **Dplyr package:** A popular package used in R which is used to solve most data manipulation problems. Some common functions that come with this package are mutate(), select(), summarise(), arrange(), and filter().
4. **Tidy Operation:** In the following model, tidy operations are performed in order for each row to only contain a single word. As the books dataset is stored in the form of phrases, the data is tidied into words.
5. **Filter():** The filter operation is performed to filter out words that meets the given condition. It can

be used to filter out words that are either positive or negative.

6. **Tokens:** In a tidy text format, tokens are used to refer to all those meaningful units for text that are to be used for analysis. Similarly, tokenization refers to the process of converting text into tokens.

8) Solved Lab Activities (Allocated Time 1 Hr. **The time can vary**)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Installing and loading the required libraries.

```
install.packages("janeaustenr")  
install.packages("stringr")  
install.packages("tidyr")  
install.packages("dplyr")  
library(janeaustenr)  
library(stringr)  
library(tidytext)  
library(dplyr)
```

Output:

Packages successfully installed.

Activity 2:

Load the “bing” lexicon

```
get_sentiments(“bing”)
```

Output:

word <chr>	sentiment <chr>
2-faces	negative
abnormal	negative
abolish	negative
abominable	negative
abominably	negative
abominate	negative
abomination	negative
abort	negative
aborted	negative
aborts	negative

Activity 3:

Convert given data into a tidy format, implementing the > operator and store inside tidy_data

```
tidy_data <- austen_books() %>%  
  group_by(book) %>%  
  mutate(linenumber = row_number(),  
         chapter = cumsum(str_detect(text, regex(“^chapter [\\divxlc]”,  
                                                ignore_case = TRUE)))) %>%  
  ungroup() %>%  
  unnest_tokens(word, text)
```

Output:

	book	linenumber	chapter	word
1	Sense & Sensibility	1	0	sense
2	Sense & Sensibility	1	0	and
3	Sense & Sensibility	1	0	sensibility
4	Sense & Sensibility	3	0	by
5	Sense & Sensibility	3	0	jane
6	Sense & Sensibility	3	0	austen

Activity 4:

Filter out and count positive sentiments from the dataset.

```
Positive_senti <- get_sentiments("bing") %>%  
  filter(sentiment == "positive")  
tidy_data %>%  
  filter(book == "Emma") %>%  
  semi_join(positive_senti) %>%  
  count(word, sort = TRUE)
```

Output:

	word	sentiment
1	abound	positive
2	abounds	positive
3	abundance	positive
4	abundant	positive
5	accessable	positive
6	accessible	positive

**9) Graded Lab Tasks((Allocated Time 1 Hr. The time can vary)
(The instructor can design the complexity and frequency of unsolved lab tasks
as per requirement of the lab. The lab tasks assigned by the instructor should
be evaluated in the same lab.)**

Lab Task 1:

Filter out negative sentiments from the dataset.

	word	sentiment
1	2-faces	negative
2	abnormal	negative
3	abolish	negative
4	abominable	negative
5	abominably	negative
6	abominate	negative

Working Code:

```
negative_senti <- get_sentiments("bing") %>%  
  filter(sentiment == "negative")
```

Lab Task 2:

Count negative sentiments from the dataset.

	<chr>	<int>
1	miss	599
2	poor	136
3	doubt	98
4	afraid	65
5	sorry	61
6	bad	60
7	cold	54
8	impossible	41
9	object	39
10	wrong	39

Lab Task 3:

Filter and count positive and negative sentiments from any other book contained in the "austen_books" dataset

Working Code:

```
tidy_data %>%  
  filter(book == "Emma") %>%  
  semi_join(negative_senti) %>%  
  count(word, sort = TRUE)
```

	word	n
	<chr>	<int>
1	good	187
2	well	163
3	great	130
4	better	87
5	like	81
6	enough	71
7	happy	64
8	best	44
9	perfectly	43
10	right	43
..	.	---

Working Code:

```
tidy_data %>%  
  filter(book == "Persuasion") %>%  
  semi_join(positive_senti) %>%  
  count(word, sort = TRUE)
```

LAB 05

Project 1

Sentimental Analysis

Objective:

The objective of this lab is to fully implement the sentimental analysis model and use basic visualization process.

Activity Outcomes:

The activities provide guidance to the following

- Analyzing both positive and negative sentiments and separating them.
- Learn to perform basic visualization process on the model
- Perform the whole process on a different lexicon

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your Rstudio

10) Useful Concepts:

1. **mutate():** The mutate function in R is used to create a new variable from a dataset.
2. **spread():** Spread() function is used to separate any given data into separate columns as it adds a new column for each unique value of the key
3. **inner_join():** The innerjoin() function is used to help merge values of rows from two dataframes having matching values.
4. **%/% or Integral Division Operator:** An operator that is used to perform basic integer division in R. I.e., 11.2 %/% 5 will return the value 2.
5. **ggplot():** ggplot is one of the most common visualizing techniques used in R. The

package must first be installed and loaded before it can be used.

11) Solved Lab Activities (Allocated Time 1 Hr. **The time can vary**)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Loading bing into an object

```
bing <- get_sentiments("bing")
```

Output:

Activity 2:

Use the spread(), mutate(), and count function to separate positive and negative sentiments.

```
Emma_sentiment <- tidy_data %>%  
  inner_join(bing) %>%  
  count(book = "Emma", index = linenumbr %/% 80, sentiment) %>%  
  spread(sentiment, n, fill = 0) %>%  
  mutate(sentiment = positive - negative)
```

Output:

Activity 3:

```
counting_words <- tidy_data %>%  
  inner_join(bing) %>%  
  count(word, sentiment, sort = TRUE)  
head(counting_words)
```

Checking the most common positive and negative words present in the dataset

Ouptut:

Activity 4:

Visualize given object using ggplot and geom_bar.

```
Ggplot(Emma_sentiment, aes(index, sentiment, fill = book)) +  
  geom_bar(stat = "identity", show.legend = TRUE) +  
  facet_wrap(~book, ncol = 2, scales = "free_x")
```

12) Graded Lab Tasks((Allocated Time 1 Hr. The time can vary)
(The instructor can design the complexity and frequency of unsolved lab tasks as per requirement of the lab. The lab tasks assigned by the instructor should be evaluated in the same lab.)

Lab Task 01:

Perform sentimental analysis on "AFFIN" lexicon.

	book	index	-5	-4	-3	-2	-1	1	2	3	4	5
1	Emma	0	-5	0	16	57	28	41	80	47	1	5
2	Emma	1	-5	0	14	65	45	37	95	59	2	5
3	Emma	2	-5	0	9	72	45	39	76	44	1	5
4	Emma	3	-5	0	12	62	46	52	120	54	2	5
5	Emma	4	-5	0	10	63	52	47	99	60	4	5
6	Emma	5	-5	0	4	53	28	45	96	73	0	5
7	Emma	6	-5	0	7	57	46	44	93	69	3	5
8	Emma	7	-5	0	7	70	47	40	99	57	2	5
9	Emma	8	-5	0	11	55	43	50	93	66	0	5
10	Emma	9	-5	0	2	41	51	39	95	68	0	5
11	Emma	10	-5	0	10	68	54	52	87	50	2	5
12	Emma	11	-5	0	11	55	30	47	77	48	1	5
13	Emma	12	-5	0	15	47	34	33	88	57	2	5
14	Emma	13	-5	0	15	62	36	57	82	50	1	5
15	Emma	14	-5	0	17	79	42	44	60	54	2	5
16	Emma	15	-5	0	4	50	43	58	76	69	2	5
17	Emma	16	-5	0	11	55	38	53	95	76	1	5
18	Emma	17	-5	0	16	68	43	47	111	68	1	5

Working Code:


```

install.packages("janeaustenr")
install.packages("stringr")
install.packages("tidytext")
install.packages("dplyr")
install.packages("textdata")
install.packages("tidyverse")
library(janeaustenr)
library(stringr)
library(tidytext)
library(dplyr)
library(textdata)
library(tidyverse)
library(ggplot2)
get_sentiments("afinn")
sentiments
tidy_data <- austen_books() %>%
  group_by(book) %>%
  mutate(linenumber = row_number(),
         chapter = cumsum(str_detect(text, regex("^chapter [\\divxlc]",
         ignore_case = TRUE)))) %>%

  ungroup() %>%
  unnest_tokens(word, text)

positive_senti <- get_sentiments("afinn") %>%
  filter(value>=0)

tidy_data %>%
  filter(book == "Emma") %>%
  semi_join(positive_senti) %>%
  count(word, sort = TRUE)

Emma_sentiment <- tidy_data %>%
  inner_join(afinn) %>%
  count(book = "Emma" , index = linenumber %/% 80, value) %>%
  spread(value, n, fill = 0) %>%
  mutate(-5,5)

```

LAB 06

Project 2

Uber Data Analysis

Objective:

The objective of this lab is to teach students how to prepare a dataset from a csv file for visualization on a date/time format.

Activity Outcomes:

The activities provide guidance to the following

- How to read a csv file
- Store datasets into objects
- Prepare and format dataset

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your Rstudio

13) Useful Concepts:

The uber data analysis project focuses on teaching more about how important data storytelling is. The model focuses on visualization concepts using ggplot2, ggthemes, and similar concepts.

1. **Lubridate:** The lubridate dataset includes dealing with time-frames. To successfully separate a data into time categories, this package is quite useful.
2. **Csv File:** A CSV file is a common file format which is often loaded as the main dataset file on which the analysis is performed.
3. **as.POSIXct:** It is an important function used in R used to store both date and time in the associated time zone, which is usually the one set on your machine.
4. **factor():** Factors are a type of data structures used in R which are mainly used to store categorical data, i.e., gender.
5. **DT:** The package is used to interface with JS libraries, also known as Datatables

i. Solved Lab Activites (Allocated Time 1 Hr. **The time can very**)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Loading the required libraries

```
library(ggplot2)
library(ggthemes)
library(lubridate)
library(dplyr)
library(tidyr)
library(DT)
library(scales)
```

Output:

Libraries successfully loaded into environment.

Activity 2:

Creating a whole vector of colors that we will be using for plotting

```
colors = c("#CC1011", "#665555", "#05a399",
"#cfcaca", "#f5e840", "#0683c9", "#e075b0")
```

Output:

```
colors = c("#CC1011", "#665555", "#05a399", "#cfcaca", "#f5e840", "#0683c9", "#e075b0")
```

Activity 3:

Downloading and reading the different csv files obtained from Uber dataset

```
apr_data <- read.csv("uber-raw-data-apr14.csv")
may_data <- read.csv("uber-raw-data-may14.csv")
jun_data <- read.csv("uber-raw-data-jun14.csv")
```

Output:

	Date.Time	Lat	Lon	Base
1	4/1/2014 0:11:00	40.7690	-73.9549	B02512
2	4/1/2014 0:17:00	40.7267	-74.0345	B02512
3	4/1/2014 0:21:00	40.7316	-73.9873	B02512
4	4/1/2014 0:28:00	40.7588	-73.9776	B02512
5	4/1/2014 0:33:00	40.7594	-73.9722	B02512
6	4/1/2014 0:33:00	40.7383	-74.0403	B02512

Activity 4:

Combining the 3 csv files into a single object

```
data_2014 <- rbind(apr_data,may_data,jun_data)
```

Output:

	Date.Time	Lat	Lon	Base	Time
1	2014-04-01 00:11:00	40.7690	-73.9549	B02512	00:11:00
2	2014-04-01 00:17:00	40.7267	-74.0345	B02512	00:17:00
3	2014-04-01 00:21:00	40.7316	-73.9873	B02512	00:21:00
4	2014-04-01 00:28:00	40.7588	-73.9776	B02512	00:28:00
5	2014-04-01 00:33:00	40.7594	-73.9722	B02512	00:33:00
6	2014-04-01 00:33:00	40.7383	-74.0403	B02512	00:33:00

Activity 5:

Formatting the columns that we require for date/time

```

data_2014$Date.Time <- as.POSIXct(data_2014$Date.Time, format =
"%m/%d/%Y %H:%M:%S")

data_2014$Time <- format(as.POSIXct(data_2014$Date.Time, format =
"%m/%d/%Y %H:%M:%S"), format="%H:%M:%S")

data_2014$Date.Time <- ymd_hms(data_2014$Date.Time)

data_2014$day <- factor(day(data_2014$Date.Time))

data_2014$month <- factor(month(data_2014$Date.Time, label = TRUE))

data_2014$year <- factor(year(data_2014$Date.Time))

data_2014$dayofweek <- factor(wday(data_2014$Date.Time, label = TRUE))

```

Output:

	Date.Time	Lat	Lon	Base	Time	day	month	year	dayofweek
1	2014-04-01 00:11:00	40.7690	-73.9549	B02512	00:11:00	1	Apr	2014	Tue
2	2014-04-01 00:17:00	40.7267	-74.0345	B02512	00:17:00	1	Apr	2014	Tue
3	2014-04-01 00:21:00	40.7316	-73.9873	B02512	00:21:00	1	Apr	2014	Tue
4	2014-04-01 00:28:00	40.7588	-73.9776	B02512	00:28:00	1	Apr	2014	Tue
5	2014-04-01 00:33:00	40.7594	-73.9722	B02512	00:33:00	1	Apr	2014	Tue
6	2014-04-01 00:33:00	40.7383	-74.0403	B02512	00:33:00	1	Apr	2014	Tue

Activity 6:

Setting a column for hours, minutes and seconds

```

data_2014$hour <- factor(hour(hms(data_2014$Time)))
data_2014$minute <- factor(minute(hms(data_2014$Time)))
data_2014$second <- factor(second(hms(data_2014$Time)))

```

Output:

	Date.Time	Lat	Lon	Base	Time	day	month	year	dayofweek	hour	minute	second
1	2014-04-01 00:11:00	40.7690	-73.9549	B02512	00:11:00	1	Apr	2014	Tue	0	11	0
2	2014-04-01 00:17:00	40.7267	-74.0345	B02512	00:17:00	1	Apr	2014	Tue	0	17	0
3	2014-04-01 00:21:00	40.7316	-73.9873	B02512	00:21:00	1	Apr	2014	Tue	0	21	0
4	2014-04-01 00:28:00	40.7588	-73.9776	B02512	00:28:00	1	Apr	2014	Tue	0	28	0
5	2014-04-01 00:33:00	40.7594	-73.9722	B02512	00:33:00	1	Apr	2014	Tue	0	33	0
6	2014-04-01 00:33:00	40.7383	-74.0403	B02512	00:33:00	1	Apr	2014	Tue	0	33	0

14) Graded Lab Tasks((Allocated Time 1 Hr. The time can vary)
(The instructor can design the complexity and frequency of unsolved lab tasks as per requirement of the lab. The lab tasks assigned by the instructor should be evaluated in the same lab.)

Task 01:

Load and implement the operations including formatting the dataset on the following csv file:

1. *uber-row-data-aug14*
2. *uber-row-data-jul14*
3. *uber-row-data-sep14*

Output:

	Date.Time	Lat	Lon	Base	Time	day	month	year	dayofweek	hour	minute	second
1	2014-08-01 00:03:00	40.7366	-73.9906	B02512	00:03:00	1	Aug	2014	Fri	0	3	0
2	2014-08-01 00:09:00	40.7260	-73.9918	B02512	00:09:00	1	Aug	2014	Fri	0	9	0
3	2014-08-01 00:12:00	40.7209	-74.0507	B02512	00:12:00	1	Aug	2014	Fri	0	12	0
4	2014-08-01 00:12:00	40.7387	-73.9856	B02512	00:12:00	1	Aug	2014	Fri	0	12	0
5	2014-08-01 00:12:00	40.7323	-74.0077	B02512	00:12:00	1	Aug	2014	Fri	0	12	0
6	2014-08-01 00:13:00	40.7349	-74.0033	B02512	00:13:00	1	Aug	2014	Fri	0	13	0

Working Code:

```
library(ggplot2)
library(ggthemes)
library(lubridate)
library(dplyr)
library(tidyr)
library(DT)
library(scales)
colors = c("#CC1011", "#665555", "#05a399", "#cfcaca", "#f5e840", "#0683c9", "#e075b0")
aug_data <- read.csv("uber-raw-data-aug14.csv")
jul_data <- read.csv("uber-raw-data-jul14.csv")
sep_data <- read.csv("uber-raw-data-sep14.csv")
data_2014 <- rbind(aug_data, jul_data, sep_data)
data_2014$Date.Time <- as.POSIXct(data_2014$Date.Time, format =
"%m/%d/%Y %H:%M:%S")
data_2014$Time <- format(as.POSIXct(data_2014$Date.Time, format =
"%m/%d/%Y %H:%M:%S"), format="%H:%M:%S")
data_2014$Date.Time <- ymd_hms(data_2014$Date.Time)
data_2014$day <- factor(day(data_2014$Date.Time))
data_2014$month <- factor(month(data_2014$Date.Time, label = TRUE))
data_2014$year <- factor(year(data_2014$Date.Time))
data_2014$dayofweek <- factor(wday(data_2014$Date.Time, label = TRUE))
data_2014$hour <- factor(hour(hms(data_2014$Time)))
data_2014$minute <- factor(minute(hms(data_2014$Time)))
data_2014$second <- factor(second(hms(data_2014$Time)))
```

LAB 07

Project 2

Uber Data Analysis

Objective:

The objective of this lab is to help students get a basic understanding of how they can perform visualization through ggplot2 for data analysis.

Activity Outcomes:

The activities provide guidance to the following

- Implement basic plotting techniques
- How to group a dataset for visualizing
- Get basic insights from visualization

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your Rstudio

15) Useful Concepts:

1. **ggplot2:** This is the most important library being used in the project, also referred to as the most popular visualization library.
2. **ggthemes:** Another popular library that is used as an add-on over the ggplot2, for the purpose of creating extra themes.
3. **geom_bar:** A popular way of introducing bar charts in analysis.
4. **x and y label:** These labels are used to denote the different axis on a visualization graph.
5. **ggtitle:** A function used to give title to a visualization graph.
6. **summarize():** It is used to create a new dataframe that can have rows depending on the combination of variables that are grouped together. In case of no grouping variables, it returns a single row that summarizes all observations.

16) Solved Lab Activities (Allocated Time 1 Hr. **The time can vary**)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Summarize the data on the basis of hour:


```
hour_data <- data_2014 %>%
  group_by(hour) %>%
  dplyr::summarize(Total = n())
datatable(hour_data)
```

Output:

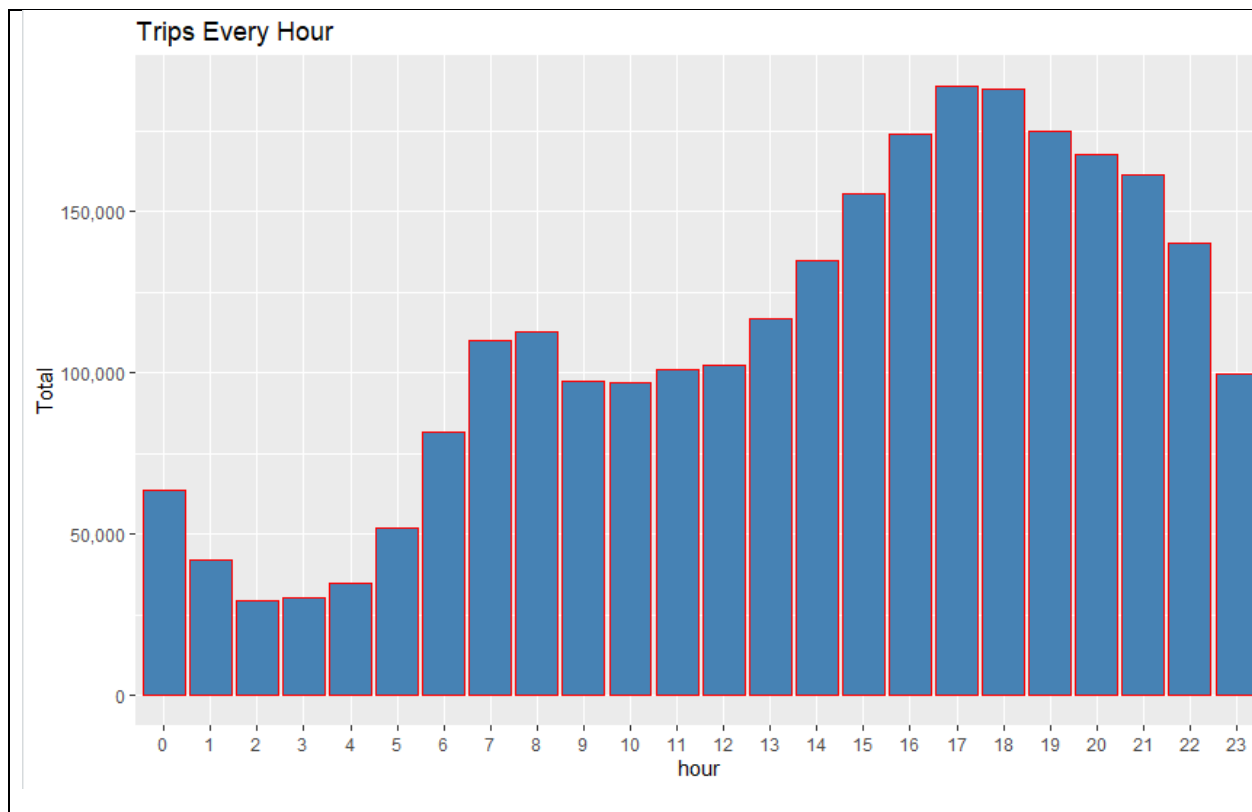
	hour	Total
1	0	63537
2	1	42105
3	2	29369
4	3	30364
5	4	34489
6	5	51719

Activity 2:

Plot a bar chart on the basis of tris every hour

```
ggplot(hour_data, aes(hour, Total)) +
  geom_bar( stat = "identity", fill = "steelblue", color = "red") +
  ggtitle("Trips Every Hour") +
  theme(legend.position = "none") +
  scale_y_continuous(labels = comma)
```

Output:



Activity 3:

Group the data on the basis of both months hour:

```
month_hour <- data_2014 %>%
  group_by(month, hour) %>%
  dplyr::summarize(Total = n())
```

Output:

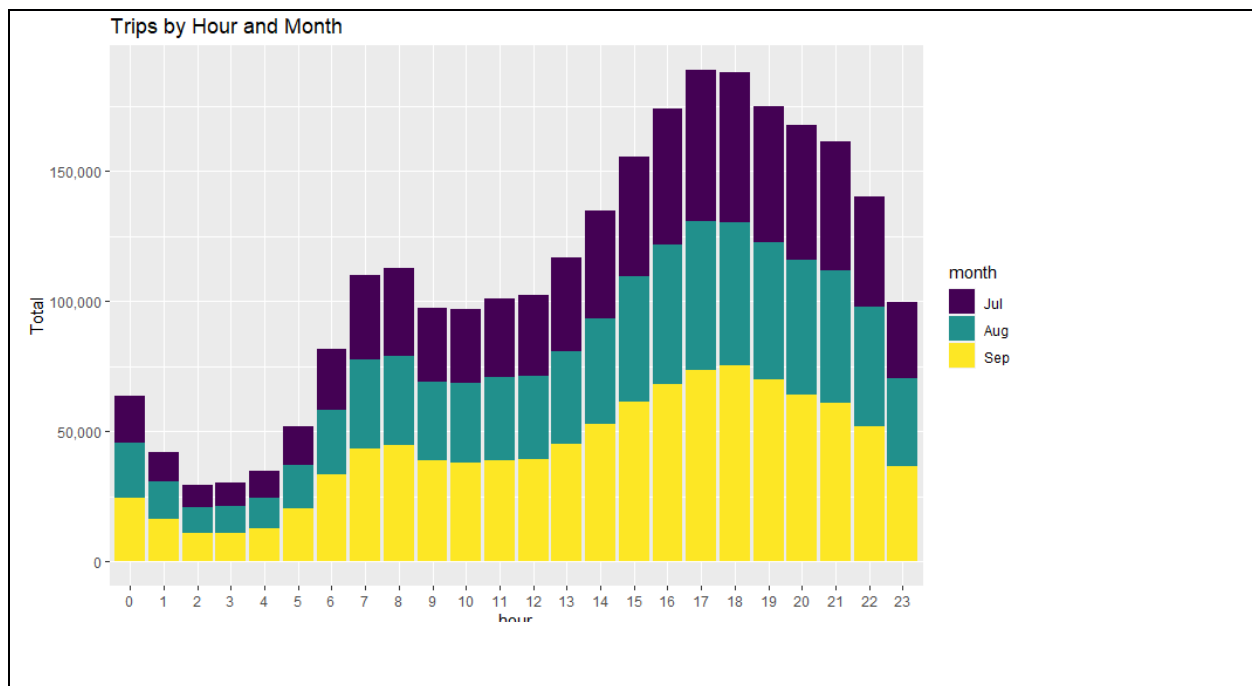
	month	hour	Total
1	Jul	0	17953
2	Jul	1	11527
3	Jul	2	8562
4	Jul	3	9199
5	Jul	4	10040
6	Jul	5	14932

Activity 4:

Visualize the data on the basis of month and hour:

```
ggplot(month_hour, aes(hour, Total, fill = month)) +  
  geom_bar( stat = "identity") +  
  ggtitle("Trips by Hour and Month") +  
  scale_y_continuous(labels = comma)
```

Output:

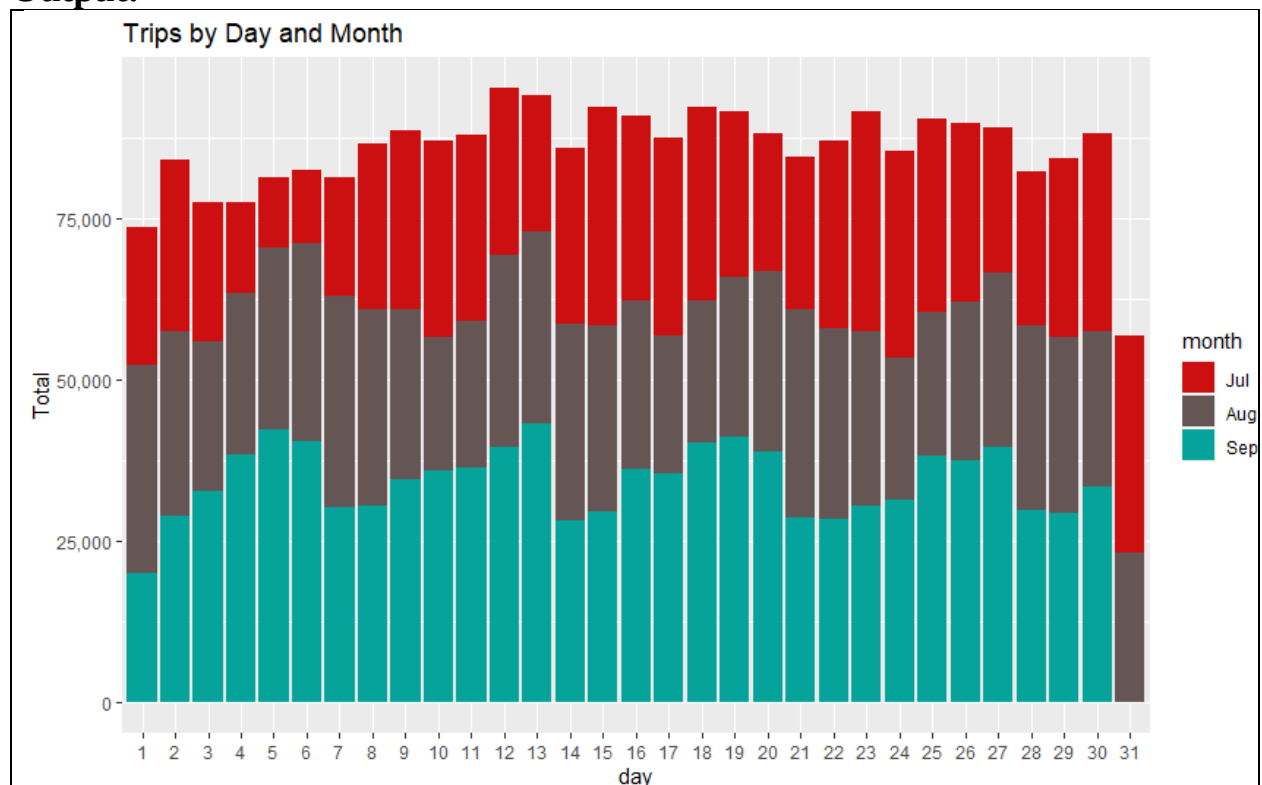


17) Graded Lab Tasks((Allocated Time 1 Hr. The time can vary)
(The instructor can design the complexity and frequency of unsolved lab tasks
as per requirement of the lab. The lab tasks assigned by the instructor should
be evaluated in the same lab.)

Task 01:

Group and visualize the dataset in a day and month format

Output:

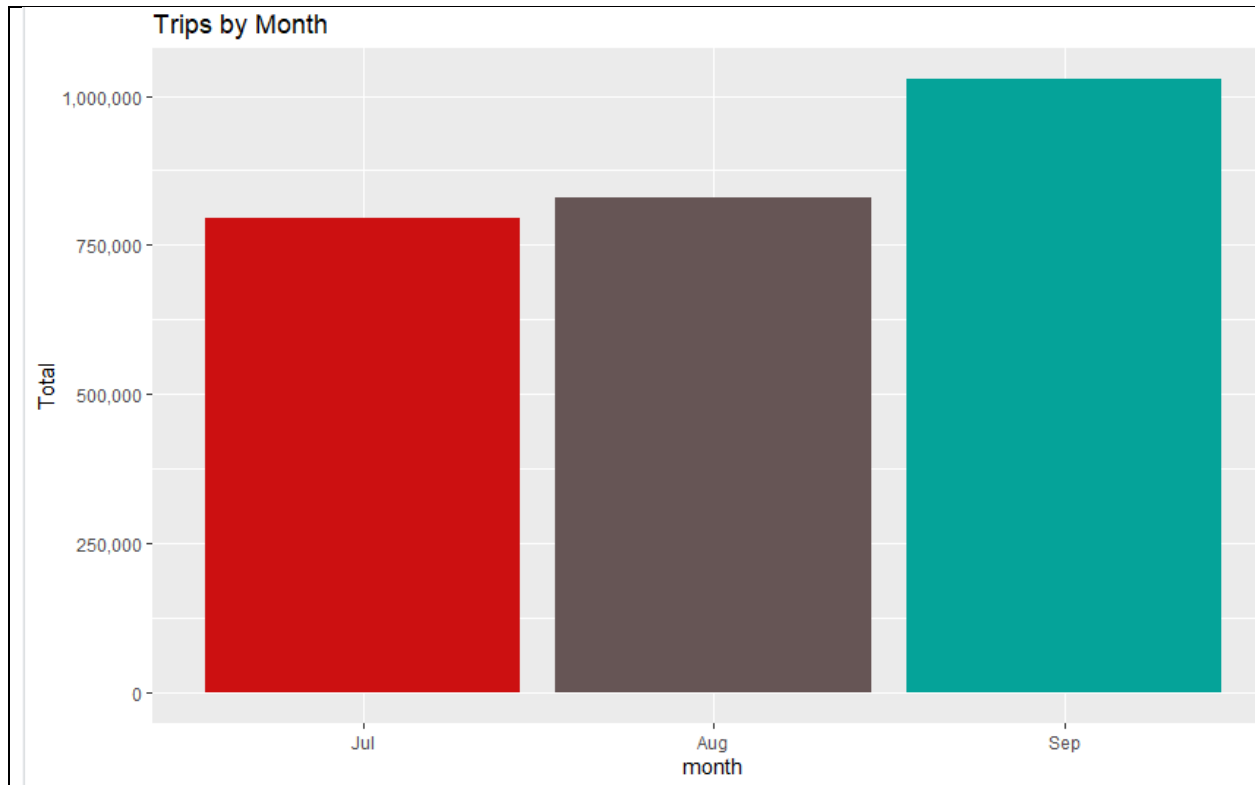


Working Code:

```
day_month_group <- data_2014 %>%
  group_by(month, day) %>%
  dplyr::summarize(Total = n())
ggplot(day_month_group, aes(day, Total, fill = month)) +
  geom_bar( stat = "identity") +
  ggtitle("Trips by Day and Month") +
  scale_y_continuous(labels = comma) +
  scale_fill_manual(values = colors)
```

Activity02:

Visualize the trips taken every month during the whole year.

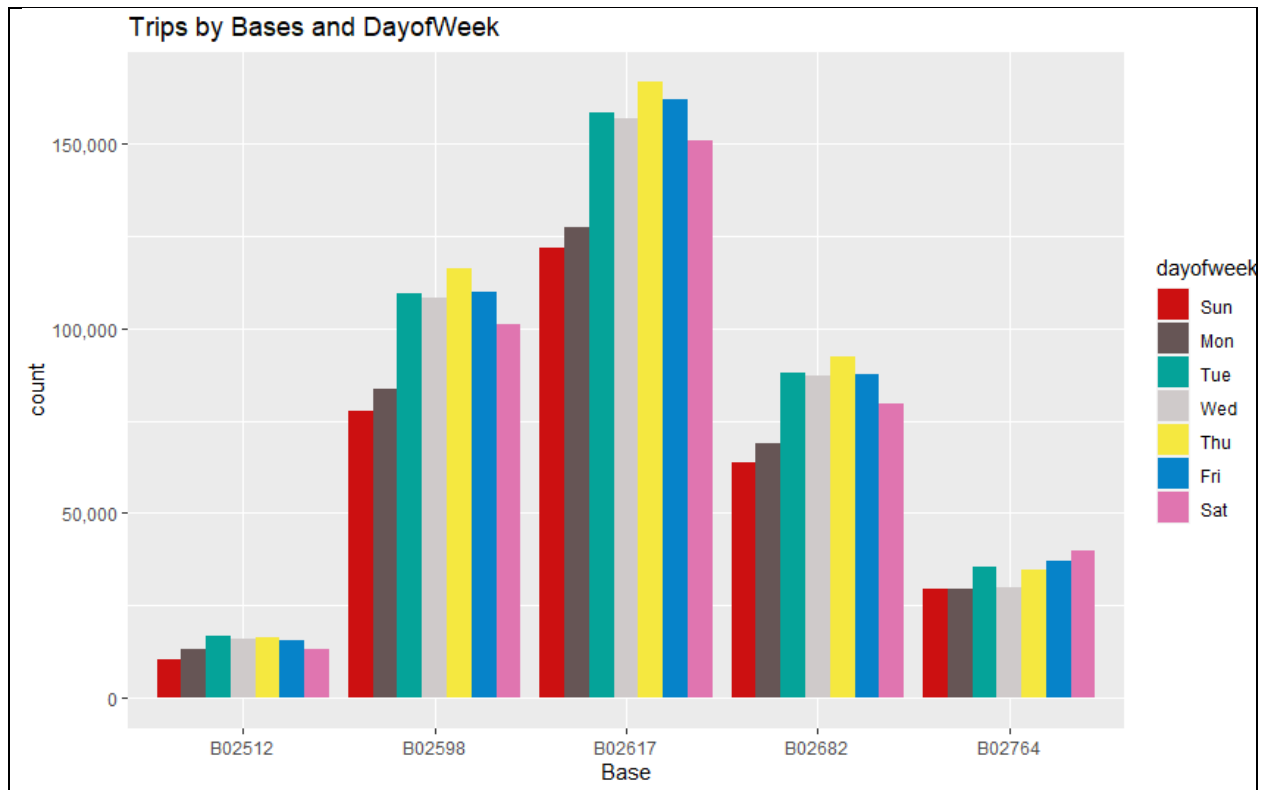


Working Code:

```
month_group <- data_2014 %>%  
  group_by(month) %>%  
  dplyr::summarize(Total = n())  
  datatable(month_group)  
  
ggplot( month_group, aes(month, Total, fill = month)) +  
  geom_bar( stat = "identity") +  
  ggtitle("Trips by Month") +  
  theme(legend.position = "none") +  
  scale_y_continuous(labels = comma) +  
  scale_fill_manual(values = colors)
```

Activity 03:

Plot the trips according to the days of week



Working Code:

```
ggplot(data_2014, aes(Base, fill = dayofweek)) +  
  geom_bar(position = "dodge") +  
  scale_y_continuous(labels = comma) +  
  ggtitle("Trips by Bases and DayofWeek") +  
  scale_fill_manual(values = colors)
```

LAB 08
Midterm Exam

Lab 09

Project 2

Uber Data Analysis

Objective:

The objective of this lab is to help students get a basic understanding of how they can perform visualization through ggplot2 for data analysis.

Activity Outcomes:

The activities provide guidance to the following

- Implement basic plotting techniques
- How to group a dataset for visualizing
- Get basic insights from visualization

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your RStudio

Useful Concepts:

7. **ggplot2:** This is the most important library being used in the project, also referred to as the most popular visualization library.
8. **ggthemes:** Another popular library that is used as an add-on over the ggplot2, for the purpose of creating extra themes.
9. **geom_bar:** A popular way of introducing bar charts in analysis.
10. **x and y label:** These labels are used to denote the different axis on a visualization graph.
11. **ggtitle:** A function used to give title to a visualization graph.
12. **summarize():** It is used to create a new dataframe that can have rows depending on the combination of variables that are grouped together. In case of no grouping variables, it returns a single row that summarizes all observations.

18) Solved Lab Activities (Allocated Time 1 Hr. **The time can vary**)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Using April, May and June Data, complete the following activities:

Activity 1:

Summarize the data on the basis of hour:

```
hour_data <- data_2014 %>%  
  group_by(hour) %>%  
  summarize(Total = n())  
datatable(hour_data)
```

Output:

Show entries

Search:

	hour	Total
1	0	40299
2	1	25122
3	2	16496
4	3	17923
5	4	20741
6	5	32220
7	6	61543
8	7	83171
9	8	78074
10	9	62744

Showing 1 to 10 of 24 entries

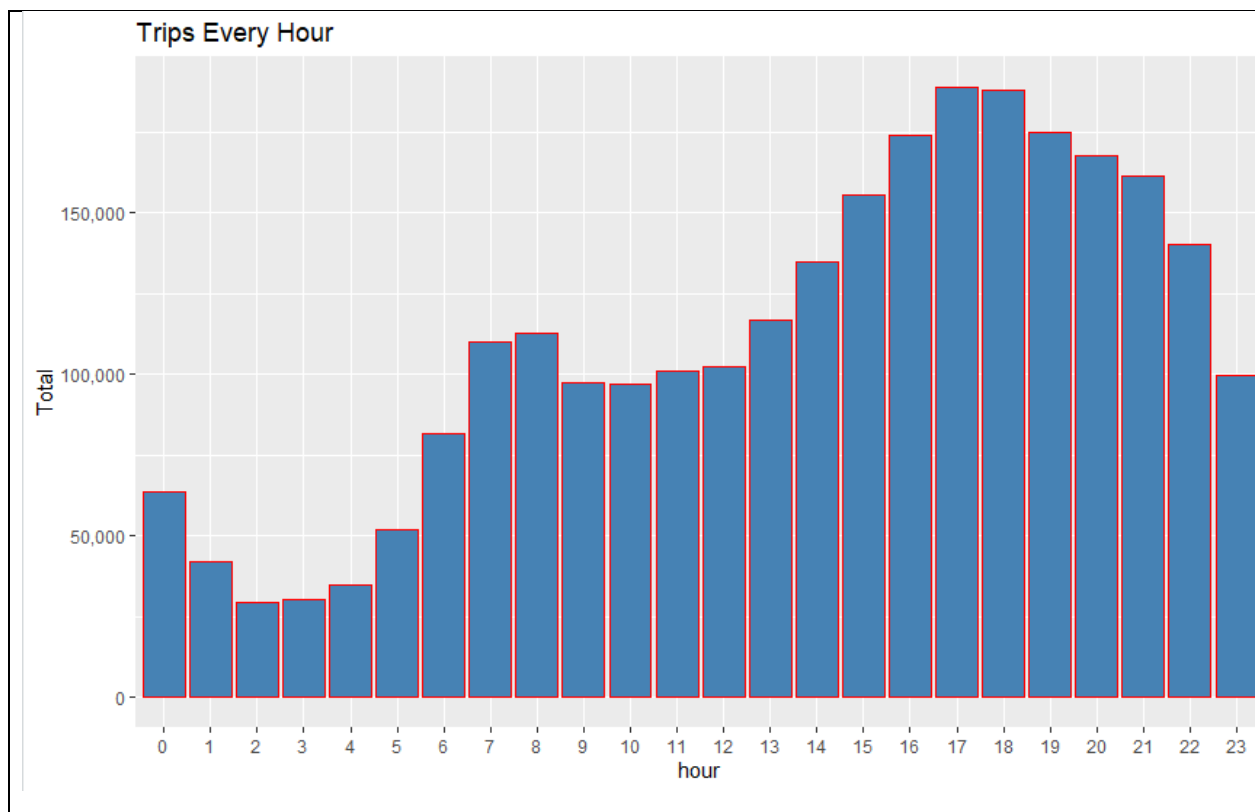
Previous 2 3 Next

Activity 2:

Plot a bar chart on the basis of trips every hour

```
ggplot(hour_data, aes(hour, Total)) +  
  geom_bar( stat = "identity", fill = "steelblue", color = "red") +  
  ggtitle("Trips Every Hour") +  
  theme(legend.position = "none") +  
  scale_y_continuous(labels = comma)
```

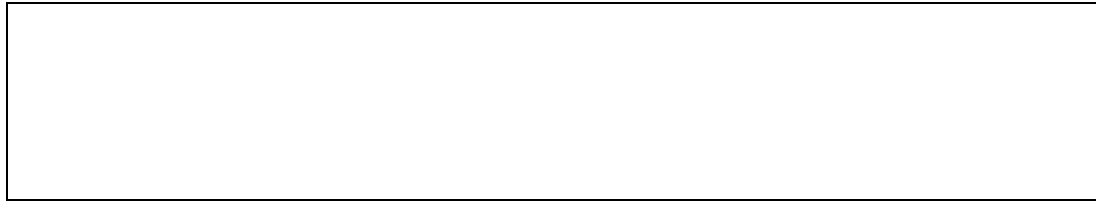
Output:



Activity 3:

Group the data on the basis of both months hour:

```
month_hour <- data_2014 %>%  
  group_by(month, hour) %>%  
  summarize(Total = n())
```



Output:

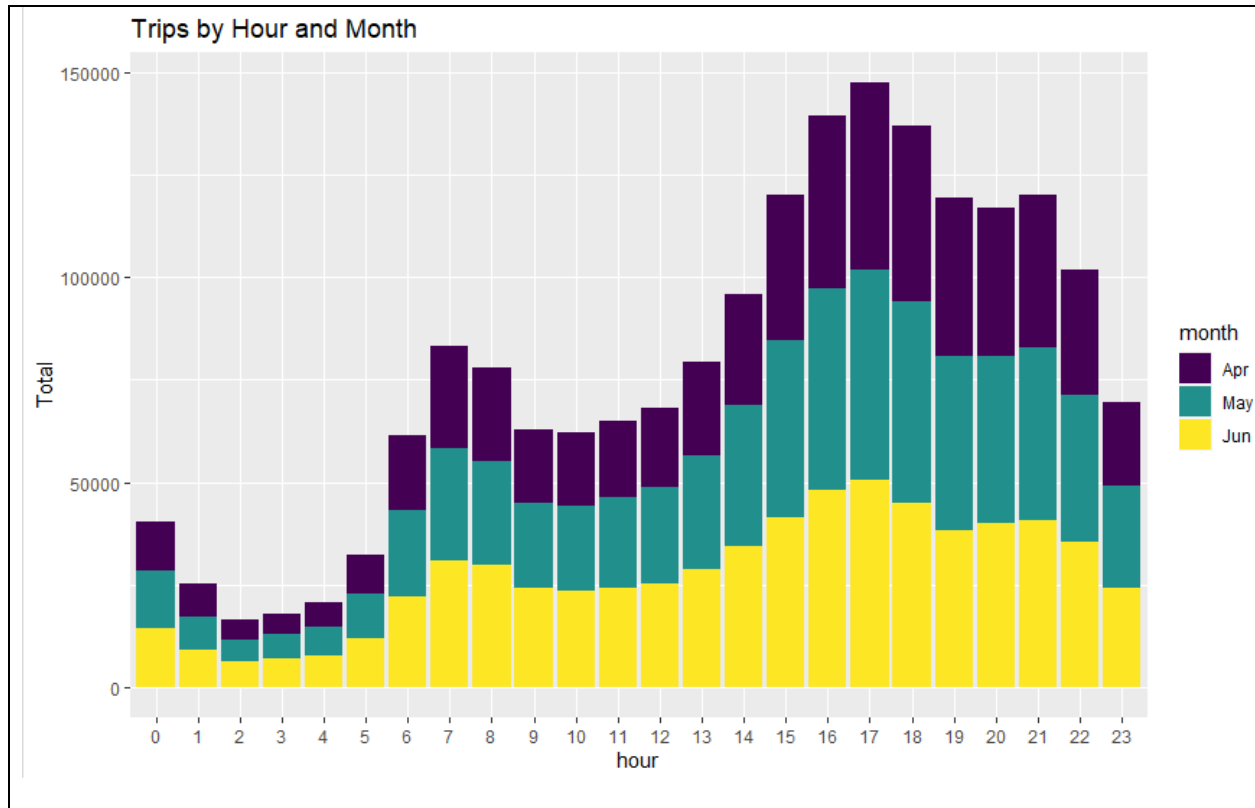
	month	hour	Total
1	Apr	0	11910
2	Apr	1	7769
3	Apr	2	4935
4	Apr	3	5040
5	Apr	4	6095
6	Apr	5	9476

Activity 4:

Visualize the data on the basis of month and hour:

```
ggplot(month_hour, aes(hour, Total, fill = month)) +  
  geom_bar( stat = "identity") +  
  ggtitle("Trips by Hour and Month") +  
  scale_y_continuous(labels = comma)
```

Output:



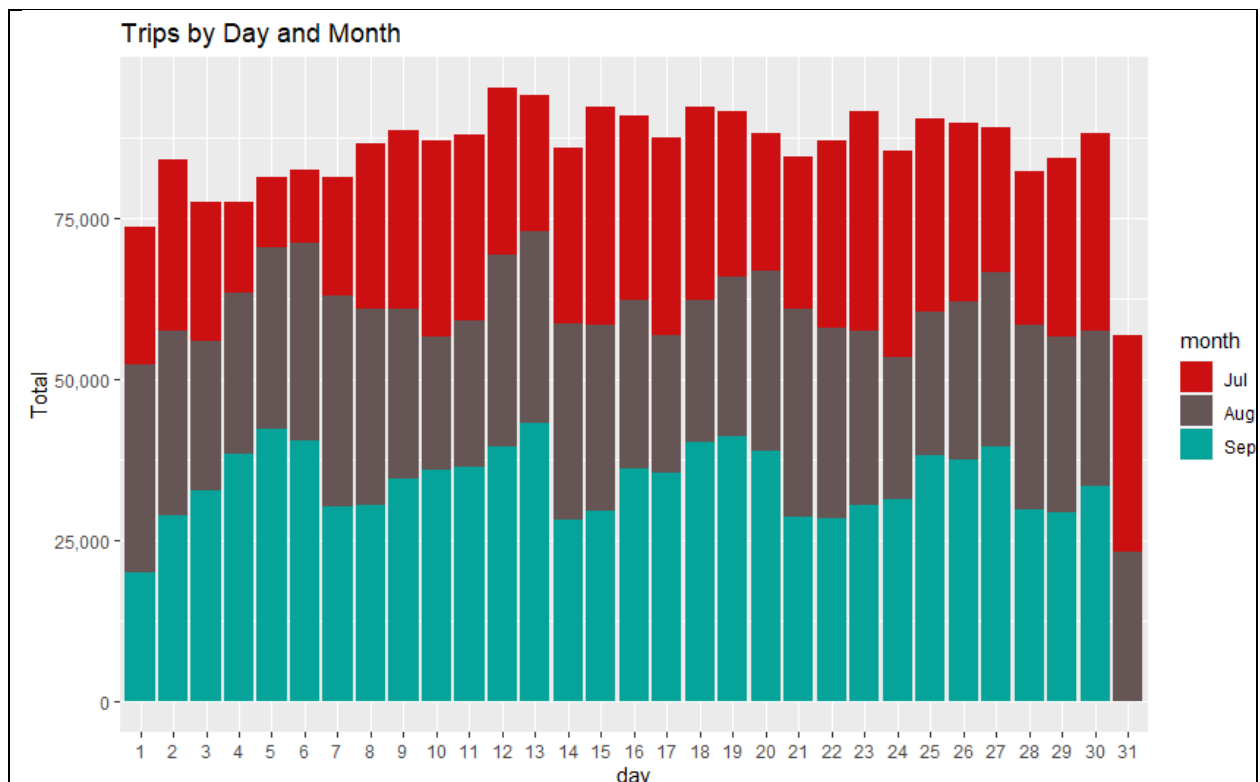
19) Graded Lab Tasks((Allocated Time 1 Hr. **The time can vary)**
(The instructor can design the complexity and frequency of unsolved lab tasks
as per requirement of the lab. The lab tasks assigned by the instructor should
be evaluated in the same lab.)

Using July, August and September Data, complete the following activities:

Task 01:

Group and visualize the dataset in a day and month format

Output:



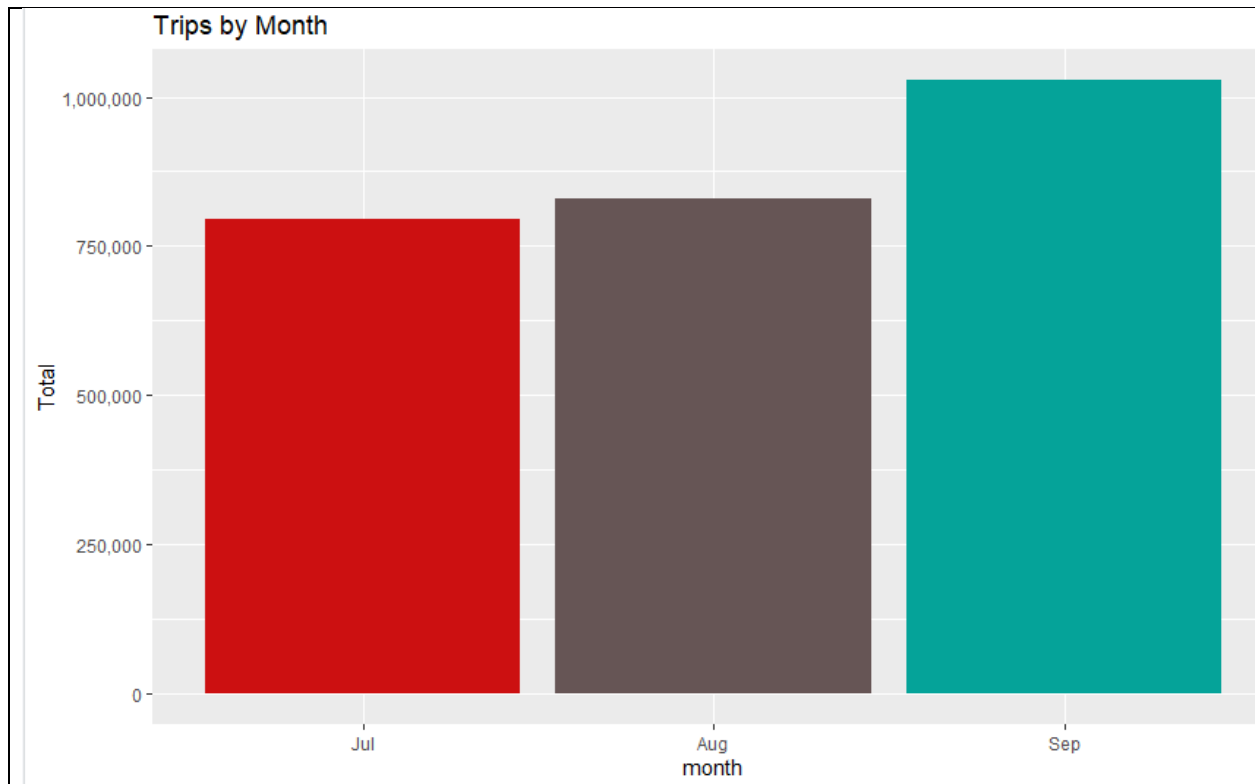
Task 1 Activity 1:

```
day_month_group <- data_2014 %>%
  group_by(month, day) %>%
  summarize(Total = n())

ggplot(day_month_group, aes(day, Total, fill = month)) +
  geom_bar( stat = "identity") +
  ggtitle("Trips by Day and Month") +
  scale_y_continuous(labels = comma) +
  scale_fill_manual(values = colors)
```

Task 2:

Visualize the trips taken every month during the whole year.



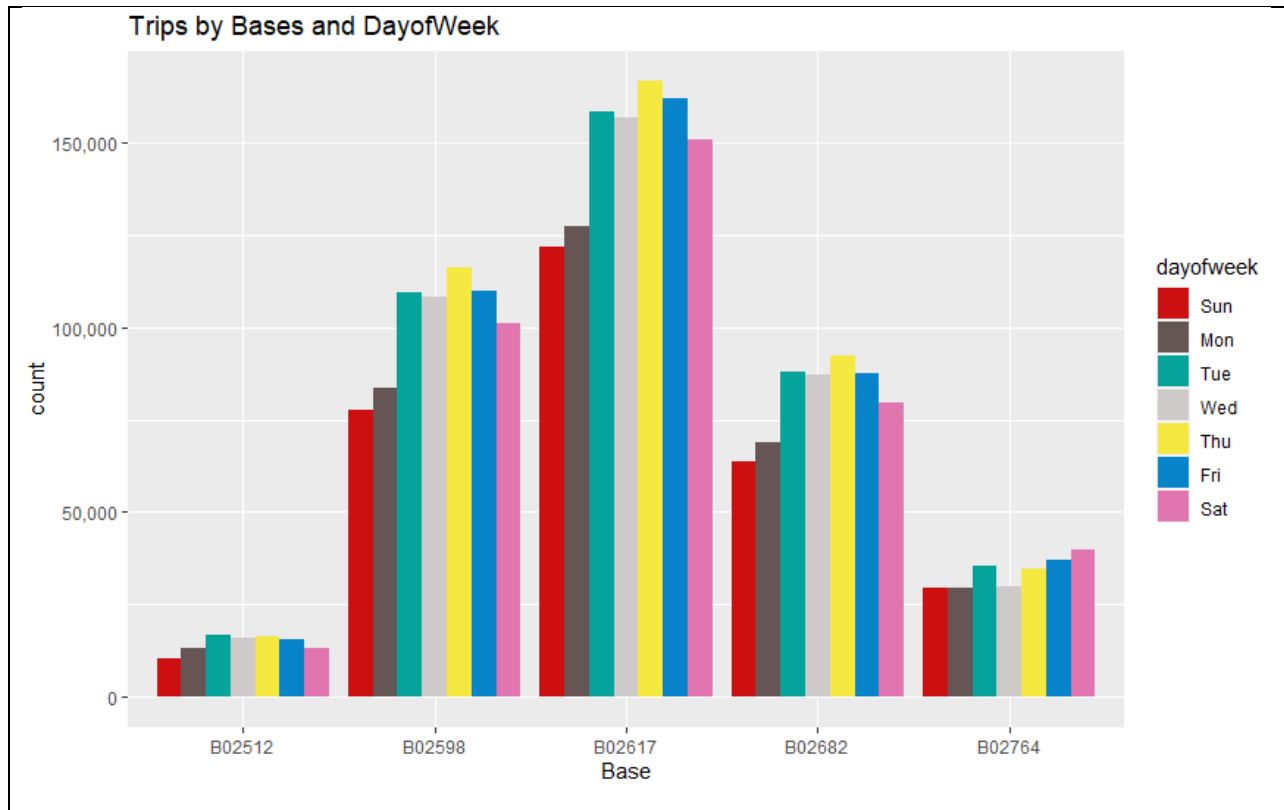
Task 2

Activity 1:

```
month_group <- data_2014 %>%  
  group_by(month) %>%  
  summarize(Total = n())  
datatable(month_group)  
  
ggplot( month_group, aes(month, Total, fill = month)) +  
  geom_bar( stat = "identity") +  
  ggtitle("Trips by Month") +  
  theme(legend.position = "none") +  
  scale_y_continuous(labels = comma) +  
  scale_fill_manual(values = colors)
```

Task 3:

Plot the trips according to the days of week



Task 3 Activity 1:

```
ggplot(data_2014, aes(Base, fill = dayofweek)) +  
  geom_bar(position = "dodge") +  
  scale_y_continuous(labels = comma) +  
  ggtitle("Trips by Bases and DayofWeek") +  
  scale_fill_manual(values = colors)
```

Lab 10

Project 3

Movie Recommendation

Objective:

The objective of this lab is to implement basic function of `recommenderlab`, `ggplot2`, `'data.table'` and `'reshape2'` to create movie recommendation system

Activity Outcomes:

The activities provide guidance to the following

- How to read a csv file
- Store datasets into objects
- Prepare and format dataset
- Create a data frame
- Convert matrix into sparse matrix

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your RStudio

20) Useful Concepts

1. *A recommendation system provides suggestions to the users through a filtering process that is based on user preferences and browsing history.*
2. You can find the `movies.csv` and `ratings.csv` file. This data consists of 105339 ratings applied over 10329 movies.
3. We will make use of these four packages – `'recommenderlab'`, `'ggplot2'`, `'data.table'` and `'reshape2'`.
4. Use the `str()` function to display information about the `movie_data` dataframe.
5. Overview the summary of the movies using the `summary()` function.
6. Convert the genres present in the `movie_data` dataframe into a more usable format by the users.

21) Solved Lab Activities (Allocated Time 1 Hr. The time can vary) (The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

R program to install packages

```
install.packages("recommenderlab")
install.packages("ggplot2")
install.packages("data.table")
install.packages("reshape2")
```

Output:

```
> install.packages("reshape2")
also installing the dependency 'plyr'

trying URL 'https://cran.rstudio.com/bin/macosx/contrib/4.1/plyr_1.8.7.tgz'
Content type 'application/x-gzip' length 1016408 bytes (992 KB)
=====
downloaded 992 KB

trying URL 'https://cran.rstudio.com/bin/macosx/contrib/4.1/reshape2_1.4.4.tgz'
Content type 'application/x-gzip' length 331915 bytes (324 KB)
=====
downloaded 324 KB

The downloaded binary packages are in
  /var/folders/4f/ycw014cn2r9bplg6vz0q6_yh0000gp/T//RtmpCUsNr0/downloaded_package
```

Activity 2:

R program to import libraries

```
library(recommenderlab)
library(ggplot2)
library(data.table)
library(reshape2)
```

Output:

```

> library(recommenderlab)
Loading required package: Matrix
Loading required package: arules

Attaching package: 'arules'

The following objects are masked from 'package:base':

  abbreviate, write

Loading required package: proxy

Attaching package: 'proxy'

The following object is masked from 'package:Matrix':

  as.matrix

The following objects are masked from 'package:stats':

  as.dist, dist

The following object is masked from 'package:base':

  as.matrix

Loading required package: registry
Registered S3 methods overwritten by 'registry':
  method      from
  print.registry_field proxy
  print.registry_entry proxy

```

Activity 3:

```

movie_data <- read.csv("movies.csv",stringsAsFactors=FALSE)
rating_data <- read.csv("ratings.csv")
str(movie_data)
summary(movie_data)
head(movie_data)
summary(rating_data)
head(rating_data)

```

Output:

```

setwd("~/Downloads/IMDB-Dataset")
movie_data <- read.csv("movies.csv",stringsAsFactors=FALSE)
rating_data <- read.csv("ratings.csv")
str(movie_data)
data.frame': 10329 obs. of 3 variables:
 $ movieId: int  1 2 3 4 5 6 7 8 9 10 ...
 $ title : chr  "Toy Story (1995)" "Jumanji (1995)" "Grumpier Old Men (1995)" "Waiting
to Exhale (1995)" ...
 $ genres : chr  "Adventure|Animation|Children|Comedy|Fantasy" "Adventure|Children|Fant
sy" "Comedy|Romance" "Comedy|Drama|Romance" ...

```

```

> summary(movie_data)
      movieId      title      genres
Min.   :    1  Length:10329  Length:10329
1st Qu.: 3240  Class :character  Class :character
Median : 7088  Mode  :character  Mode  :character
Mean   : 31924
3rd Qu.: 59900
Max.   :149532
> head(movie_data)
      movieId      title
1         1      Toy Story (1995)
2         2      Jumanji (1995)
3         3  Grumpier Old Men (1995)
4         4  Waiting to Exhale (1995)
5         5 Father of the Bride Part II (1995)
6         6      Heat (1995)
      genres
1 Adventure|Animation|Children|Comedy|Fantasy
2      Adventure|Children|Fantasy
3      Comedy|Romance
4      Comedy|Drama|Romance
5      Comedy
6      Action|Crime|Thriller
> summary(rating_data)
      userId      movieId      rating      timestamp
Min.   : 1.0  Min.   :    1  Min.   :0.500  Min.   :8.286e+08
1st Qu.:192.0 1st Qu.: 1073 1st Qu.:3.000 1st Qu.:9.711e+08
Median :383.0 Median : 2497 Median :3.500 Median :1.115e+09
Mean   :364.9 Mean   : 13381 Mean   :3.517 Mean   :1.130e+09
3rd Qu.:557.0 3rd Qu.: 5991 3rd Qu.:4.000 3rd Qu.:1.275e+09
Max.   :668.0 Max.   :149532 Max.   :5.000 Max.   :1.452e+09

```

```

head(rating_data)
userId movieId rating timestamp
1      16      4.0 1217897793
1      24      1.5 1217895807
1      32      4.0 1217896246
1      47      4.0 1217896556
1      50      4.0 1217896523
1     110      4.0 1217896150

```

Activity 4:

R program to create a data frame including both movieID and userID

```
movie_genre <- as.data.frame(movie_data$genres, stringsAsFactors=FALSE)
library(data.table)
movie_genre2 <- as.data.frame(tstrsplit(movie_genre[,1], '[|]',
                                     type.convert=TRUE),
                             stringsAsFactors=FALSE) #DataFlair
colnames(movie_genre2) <- c(1:10)

list_genre <- c("Action", "Adventure", "Animation", "Children",
               "Comedy", "Crime", "Documentary", "Drama", "Fantasy",
               "Film-Noir", "Horror", "Musical", "Mystery", "Romance",
               "Sci-Fi", "Thriller", "War", "Western")
genre_mat1 <- matrix(0,10330,18)
genre_mat1[1,] <- list_genre
colnames(genre_mat1) <- list_genre

for (index in 1:nrow(movie_genre2)) {
  for (col in 1:ncol(movie_genre2)) {
    gen_col = which(genre_mat1[1,] == movie_genre2[index,col]) #Author DataFlair
    genre_mat1[index+1,gen_col] <- 1
  }
}
genre_mat2 <- as.data.frame(genre_mat1[-1,], stringsAsFactors=FALSE) #remove first row,
which was the genre list
for (col in 1:ncol(genre_mat2)) {
  genre_mat2[,col] <- as.integer(genre_mat2[,col]) #convert from characters to integers
}
str(genre_mat2)
```

Output:

▶ genre_mat2	10329 obs. of 18 variables
▶ movie_data	10329 obs. of 3 variables
▶ movie_genre	10329 obs. of 1 variable
▶ movie_genre2	10329 obs. of 10 variables
▶ rating_data	105339 obs. of 4 variables

```
> str(genre_mat2)
'data.frame': 10329 obs. of 18 variables:
 $ Action      : int  0 0 0 0 0 1 0 0 1 1 ...
 $ Adventure   : int  1 1 0 0 0 0 0 1 0 1 ...
 $ Animation   : int  1 0 0 0 0 0 0 0 0 0 ...
 $ Children    : int  1 1 0 0 0 0 0 1 0 0 ...
 $ Comedy      : int  1 0 1 1 1 0 1 0 0 0 ...
 $ Crime       : int  0 0 0 0 0 1 0 0 0 0 ...
 $ Documentary : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Drama       : int  0 0 0 1 0 0 0 0 0 0 ...
 $ Fantasy     : int  1 1 0 0 0 0 0 0 0 0 ...
 $ Film-Noir   : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Horror      : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Musical     : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Mystery     : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Romance     : int  0 0 1 1 0 0 1 0 0 0 ...
 $ Sci-Fi      : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Thriller    : int  0 0 0 0 0 1 0 0 0 1 ...
 $ War         : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Western     : int  0 0 0 0 0 0 0 0 0 0 ...
```

22) Graded Lab Tasks((Allocated Time 1 Hr. **The time can vary)**
(The instructor can design the complexity and frequency of unsolved lab tasks
as per requirement of the lab. The lab tasks assigned by the instructor should
be evaluated in the same lab.)

Lab task 1:

Create a 'search matrix' that will allow us to perform an easy search of the films by specifying the genre present in our list

Working Code:

```
SearchMatrix <- cbind(movie_data[,1:2], genre_mat2[])
head(SearchMatrix)
```

```
##      movieId                                title Action Adventure Animation
## 1         1                        Toy Story (1995)      0          1          1
## 2         2                        Jumanji (1995)      0          1          0
## 3         3          Grumpier Old Men (1995)      0          0          0
## 4         4          Waiting to Exhale (1995)      0          0          0
## 5         5 Father of the Bride Part II (1995)      0          0          0
## 6         6                        Heat (1995)       1          0          0
##      Children Comedy Crime Documentary Drama Fantasy Film-Noir Horror Musical
## 1         1         1     0           0     0         1          0          0     0
## 2         1         0     0           0     0         1          0          0     0
## 3         0         1     0           0     0         0          0          0     0
## 4         0         1     0           0     1         0          0          0     0
## 5         0         1     0           0     0         0          0          0     0
## 6         0         0     1           0     0         0          0          0     0
##      Mystery Romance Sci-Fi Thriller War Western
```

Lab task 2:

Convert rating matrix into a recommenderlab sparse matrix and create recommendation model.

Working Code:

```
ratingMatrix <- dcast(rating_data, userId~movieId, value.var = "rating", na.rm=FALSE)
ratingMatrix <- as.matrix(ratingMatrix[,-1]) #remove userIds
#Convert rating matrix into a recommenderlab sparse matrix
ratingMatrix <- as(ratingMatrix, "realRatingMatrix")
ratingMatrix

recommendation_model <- recommenderRegistry$get_entries(dataType = "realRatingMatrix")
names(recommendation_model)

lapply(recommendation_model, "[", "description")
recommendation_model$IBCF_realRatingMatrix$parameters
```

```
## 668 x 10325 rating matrix of class 'realRatingMatrix' with 105339 ratings.
```

Lab task 3:

Delineate the similarity that is shared between the films and the similarity that is shared between the users

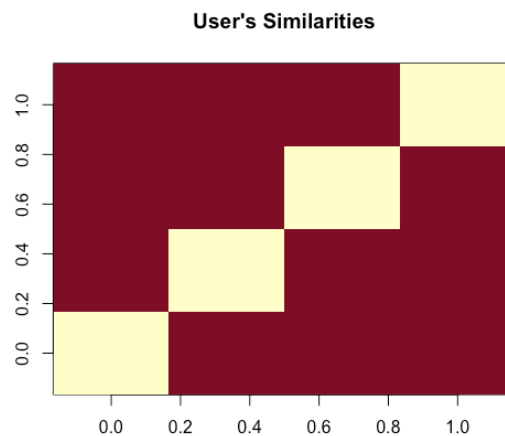
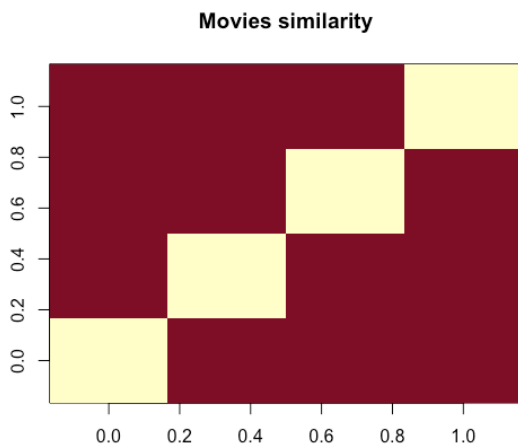
Working Code:

```
similarity_mat <- similarity(ratingMatrix[1:4, ],
                             method = "cosine",
                             which = "users")
as.matrix(similarity_mat)
image(as.matrix(similarity_mat), main = "User's Similarities")

movie_similarity <- similarity(ratingMatrix[, 1:4], method =
                              "cosine", which = "items")
as.matrix(movie_similarity)
image(as.matrix(movie_similarity), main = "Movies similarity")
```

```
1 0.0000000 0.9760860 0.9641723 0.9914398
2 0.9760860 0.0000000 0.9925732 0.9374253
3 0.9641723 0.9925732 0.0000000 0.9888968
4 0.9914398 0.9374253 0.9888968 0.0000000
> image(as.matrix(similarity_mat), main = "User's Similarities")
> movie_similarity <- similarity(ratingMatrix[, 1:4], method =
+                               "cosine", which = "items")
> as.matrix(movie_similarity)
```

	1	2	3	4
1	0.0000000	0.9669732	0.9559341	0.9101276
2	0.9669732	0.0000000	0.9658757	0.9412416
3	0.9559341	0.9658757	0.0000000	0.9864877
4	0.9101276	0.9412416	0.9864877	0.0000000



Lab 11

Project 3

Movie Recommendation

Objective:

The objective of this lab is to binarize data and visualize the top users and movies.

Activity Outcomes:

The activities provide guidance to the following

- Selecting useful data.
- Normalizing data.
- Binarizing the data.
- Visualize a heatmap

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your RStudio

23) Useful Concepts

1. Collaborative Filtering involves suggesting movies to the users that are based on collecting preferences from many other users. For example, if a user A likes to watch action films and so does user B, then the movies that the user B will watch in the future will be recommended to A and vice-versa.
2. We will conduct data preparation in the following three steps –
 - Selecting useful data.
 - Normalizing data.
 - Binarizing the data.
3. For finding useful data in our dataset, we have set the threshold for the minimum number of users who have rated a film as 50. This is also same for minimum number of views that are per film.
4. Binarizing the data means that we have two discrete values 1 and 0, which will allow our recommendation systems to work more efficiently. We will define a matrix that will consist of 1 if the rating is above 3 and otherwise it will be 0

10) Solved Lab Activities (Allocated Time 1 Hr. **The time can vary**)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Extracting unique ratings and creating a count of movie ratings

```
rating_values <- as.vector(ratingMatrix@data)
unique(rating_values) # extracting unique ratings

Table_of_Ratings <- table(rating_values) # creating a count of movie ratings
Table_of_Ratings
```

Output:

```
> unique(rating_values) # extracting unique ratings
[1] 0.0 5.0 4.0 3.0 4.5 1.5 2.0 3.5 1.0 2.5 0.5
> Table_of_Ratings <- table(rating_values) # creating a count of movie ratings
> Table_of_Ratings
rating_values
  0      0.5      1      1.5      2      2.5      3      3.5      4      4.5
5791761 1198    3258    1567    7943    5484    21729    12237    28880    8187
  5
14856
```

Activity 2:

Count views for each movie and create data frame of view, sort by number of views

```
library(ggplot2)
movie_views <- colCounts(ratingMatrix) # count views for each movie
table_views <- data.frame(movie = names(movie_views),
                          views = movie_views) # create dataframe of views
table_views <- table_views[order(table_views$views,
                                decreasing = TRUE), ] # sort by number of views
table_views$title <- NA
for (index in 1:10325){
  table_views[index,3] <- as.character(subset(movie_data,
                                              movie_data$movieid == table_views[index,1])$title)
}
```

Output:

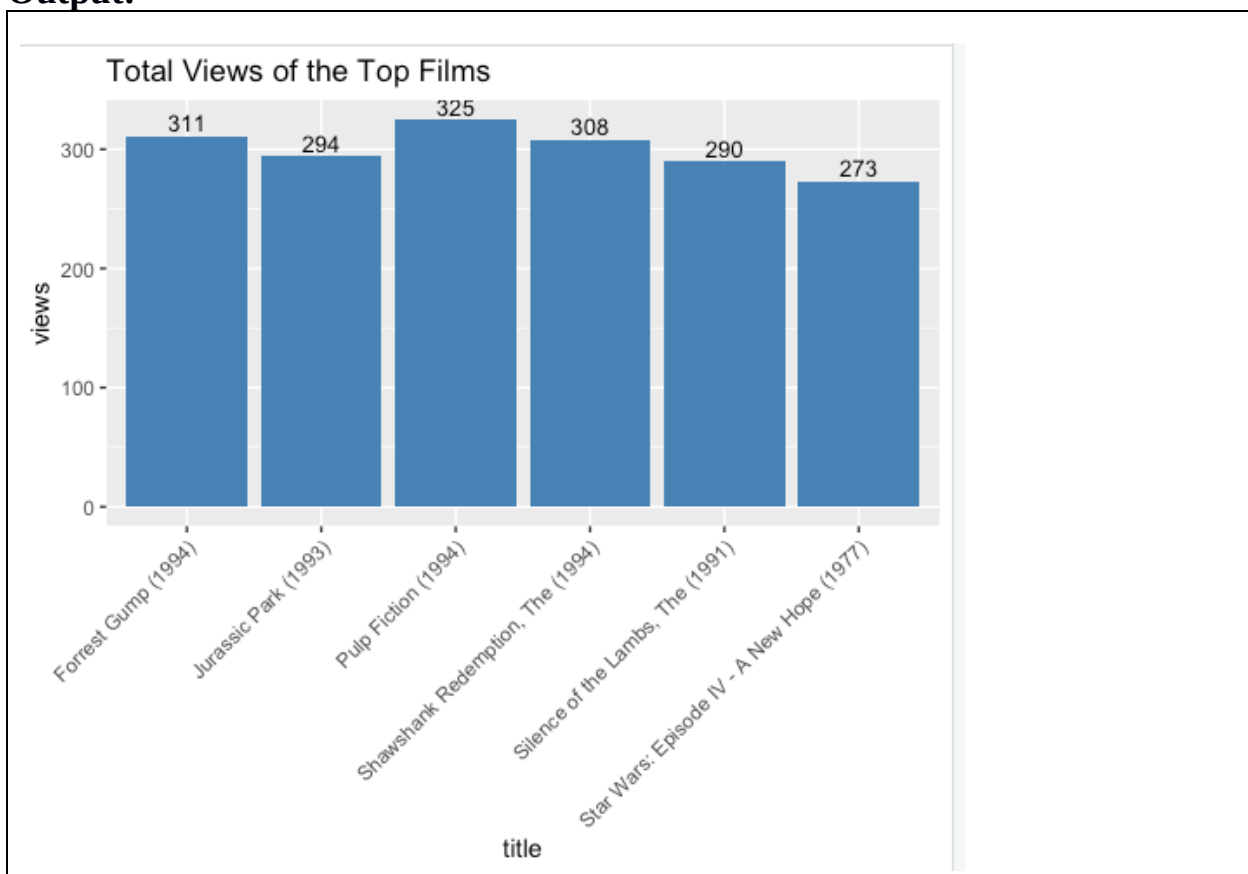
movie views			title
296	296	325	Pulp Fiction (1994)
356	356	311	Forrest Gump (1994)
318	318	308	Shawshank Redemption, The (1994)
480	480	294	Jurassic Park (1993)
593	593	290	Silence of the Lambs, The (1991)
260	260	273	Star Wars: Episode IV - A New Hope (1977)

Activity 3:

Plot total views of the top films

```
ggplot(table_views[1:6, ], aes(x = title, y = views)) +
  geom_bar(stat="identity", fill = 'steelblue') +
  geom_text(aes(label=views), vjust=-0.3, size=3.5) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  ggtitle("Total Views of the Top Films")
```

Output:



11) Graded Lab Tasks((Allocated Time 1 Hr. The time can vary)
(The instructor can design the complexity and frequency of unsolved lab tasks
as per requirement of the lab. The lab tasks assigned by the instructor should
be evaluated in the same lab.)

Lab task 1:

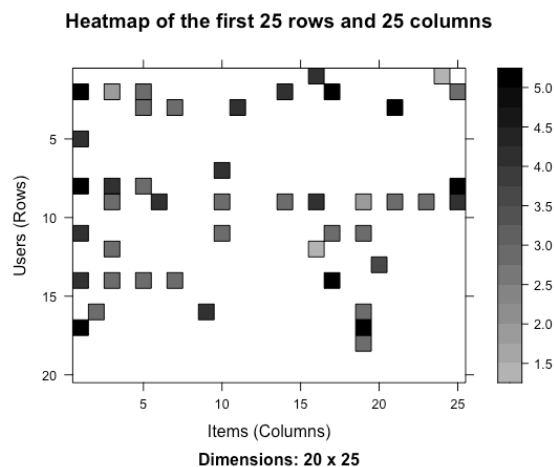
Visualize a heatmap of the movie ratings.

Working Code:

```
image(ratingMatrix[1:20, 1:25], axes = FALSE, main = "Heatmap of the first 25 rows and 25 columns")

movie_ratings <- ratingMatrix[rowCounts(ratingMatrix) > 50,
                             colCounts(ratingMatrix) > 50]

movie_ratings
```



Output:

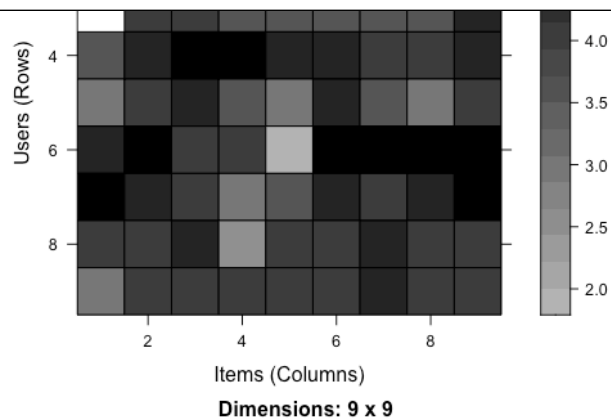
```
420 x 447 rating matrix of class 'realRatingMatrix' with 38341 ratings.
```

Lab task 2:

Visualize a heatmap of the top users and movies.

Working Code:

```
minimum_movies<- quantile(rowCounts(movie_ratings), 0.98)
minimum_users <- quantile(colCounts(movie_ratings), 0.98)
image(movie_ratings[rowCounts(movie_ratings) > minimum_movies,
          colCounts(movie_ratings) > minimum_users],
      main = "Heatmap of the top users and movies")
```



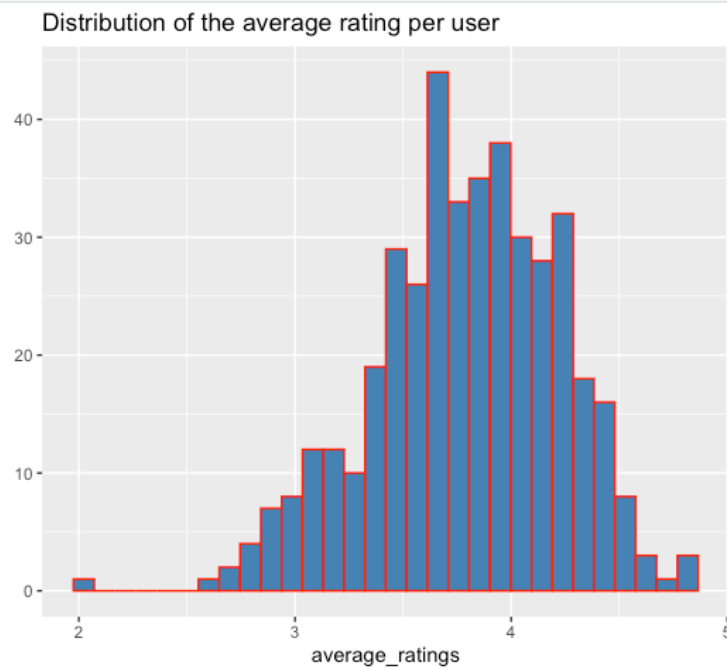
Lab task 3:

Visualize distribution of the average ratings and normalize ratings.

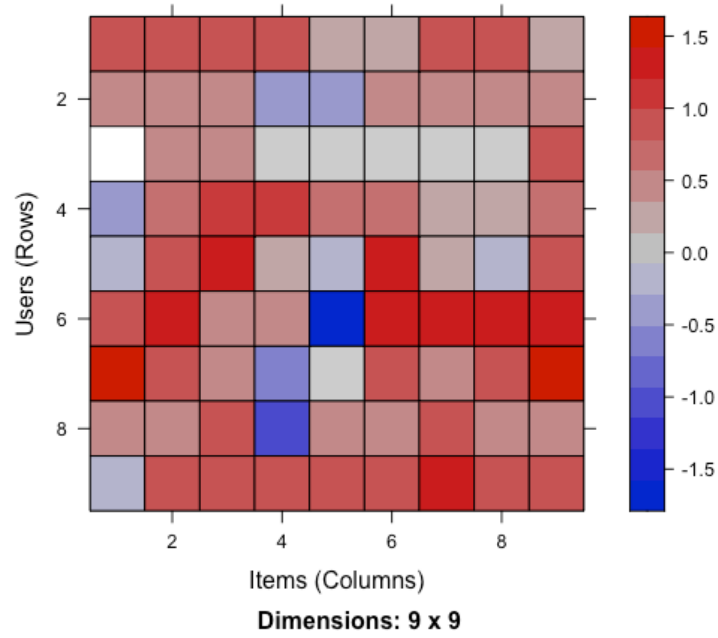
Working Code:

```
average_ratings <- rowMeans(movie_ratings)
qplot(average_ratings, fill=l("steelblue"), col=l("red")) +
  ggtitle("Distribution of the average rating per user")

normalized_ratings <- normalize(movie_ratings)
sum(rowMeans(normalized_ratings) > 0.00001)
image(normalized_ratings[rowCounts(normalized_ratings) >
  minimum_movies,
  colCounts(normalized_ratings) > minimum_users],
  main = "Normalized Ratings of the Top Users")
```



Normalized Ratings of the Top Users



Lab 12

Project 3

Movie Recommendation

Objective:

The objective of this lab is to train, test and predict model .

Activity Outcomes:

The activities provide guidance to the following

- Training the model
- Testing the model
- Predicting the model.

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your RStudio

24) Useful Concepts

1. k denotes the number of items for computing their similarities. Here, k is equal to 30. Therefore, the algorithm will now identify the k most similar items and store their number.
2. Using the getModel() function, we will retrieve the recommen_model. We will then find the class and dimensions of our similarity matrix that is contained within model_info. Finally, we will generate a heatmap, that will contain the top 20 items and visualize the similarity shared between them.
3. In the next step of ML project, we will carry out the sum of rows and columns with the similarity of the objects above 0. We will visualize the sum of columns through a distribution
4. predict() function that will identify similar items and will rank them appropriately. Here, each rating is used as a weight. Each weight is multiplied with related similarities.

25) Solved Lab Activites (Allocated Time 1 Hr. **The time can vary**)

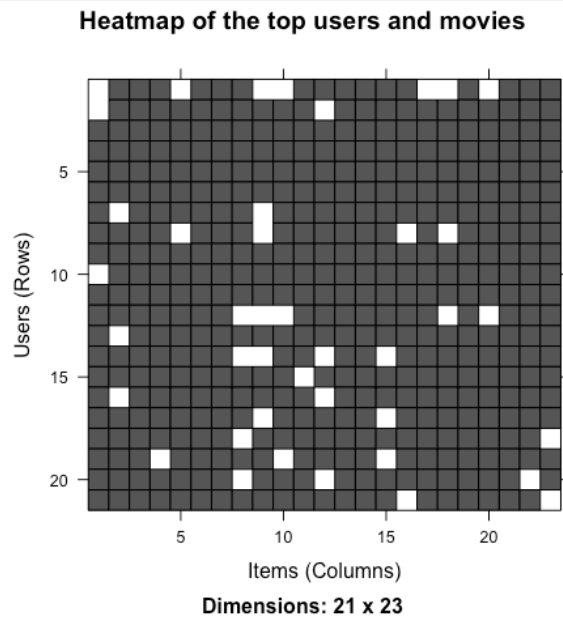
(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Binarize the data and create heatmap of top users and movies

```
binary_minimum_movies <- quantile(rowCounts(movie_ratings), 0.95)
binary_minimum_users <- quantile(colCounts(movie_ratings), 0.95)
#movies_watched <- binarize(movie_ratings, minRating = 1)
goodRatedFilms <- binarize(movie_ratings, minRating = 3)
image(goodRatedFilms[rowCounts(movie_ratings) > binary_minimum_movies,
colCounts(movie_ratings) > binary_minimum_users],
main = "Heatmap of the top users and movies")
```

Output:



Activity 2:

Perform training and testing on model.

```
sampled_data<- sample(x = c(TRUE, FALSE),
                      size = nrow(movie_ratings),
                      replace = TRUE,
                      prob = c(0.8, 0.2))
training_data <- movie_ratings[sampled_data, ]
testing_data <- movie_ratings[!sampled_data, ]

recommendation_system <- recommenderRegistry$get_entries(dataType
="realRatingMatrix")
recommendation_system$IBCF_realRatingMatrix$parameters
recommen_model <- Recommender(data = training_data,
                              method = "IBCF",
                              parameter = list(k = 30))
recommen_model
class(recommen_model)
```

Output:

```
$k
[1] 30

$method
[1] "Cosine"

$normalize
[1] "center"

$normalize_sim_matrix
[1] FALSE

$alpha
[1] 0.5

$na_as_zero
[1] FALSE
```

▶ testing_data	Formal class 'realRatingMatrix'
▶ training_data	Formal class 'realRatingMatrix'

```
> recommen_model
Recommender of type 'IBCF' for 'realRatingMatrix'
learned using 334 users.
> class(recommen_model)
[1] "Recommender"
attr(,"package")
[1] "recommenderlab"
```

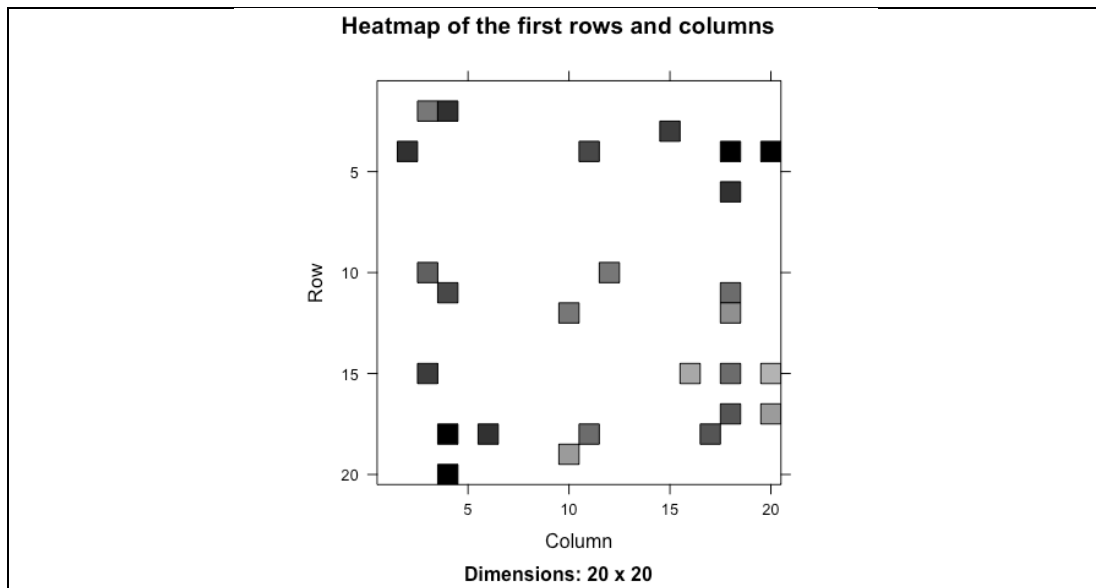
26) Graded Lab Tasks((Allocated Time 1 Hr. **The time can vary)**
(The instructor can design the complexity and frequency of unsolved lab tasks
as per requirement of the lab. The lab tasks assigned by the instructor should
be evaluated in the same lab.)

Lab task 1:

Using the getModel() function, retrieve the recommen_model. find the class and dimensions of our similarity matrix that is contained within model_info, generate a heatmap, that will contain the top 20 items and visualize the similarity shared between them.

Working Code:

```
model_info <- getModel(recommen_model)
class(model_info$sim)
dim(model_info$sim)
top_items <- 20
image(model_info$sim[1:top_items, 1:top_items],
      main = "Heatmap of the first rows and columns")
```

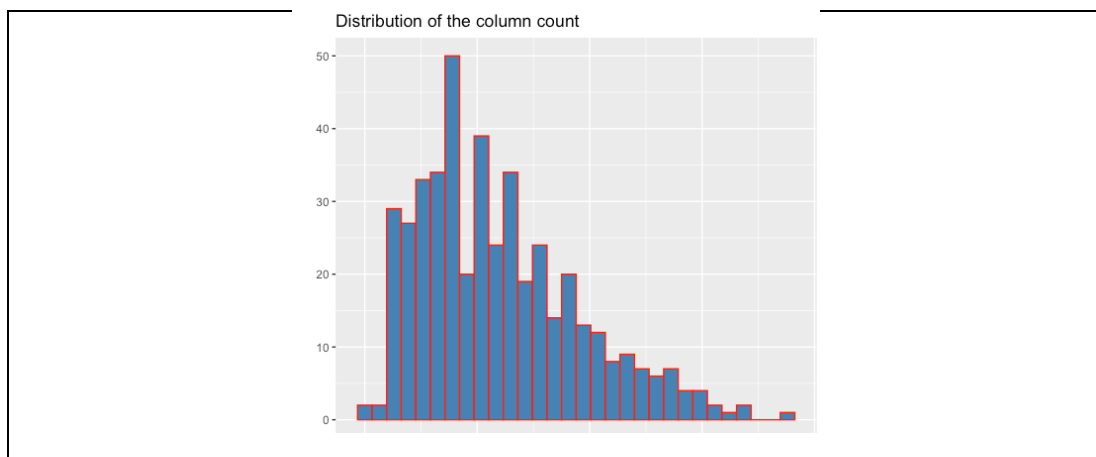


Lab task 2:

Carry out the sum of rows and columns with the similarity of the objects above 0. visualize the sum of columns through a distribution

Working Code:

```
sum_rows <- rowSums(model_info$sim > 0)
table(sum_rows)
sum_cols <- colSums(model_info$sim > 0)
qplot(sum_cols, fill=l("steelblue"), col=l("red"))+ ggtitle("Distribution of the column count")
```



Lab task 3:

Use the predict() function that will identify similar items and will rank them appropriately

```
top_recommendations <- 10 # the number of items to recommend to each user
predicted_recommendations <- predict(object = recommen_model,
                                     newdata = testing_data,
                                     n = top_recommendations)
predicted_recommendations

user1 <- predicted_recommendations@items[[1]] # recommendation for the first user
movies_user1 <- predicted_recommendations@itemLabels[user1]
movies_user2 <- movies_user1
for (index in 1:10){
  movies_user2[index] <- as.character(subset(movie_data,
                                             movie_data$movieid == movies_user1[index])$title)
}
movies_user2
```

```
> recommendation_matrix[,1:4]
```

	[,1]	[,2]	[,3]	[,4]
[1,]	2291	70	1	7
[2,]	3000	208	5	11
[3,]	1203	497	16	39
[4,]	48394	508	17	153
[5,]	1225	745	21	158
[6,]	6502	1148	25	185
[7,]	2692	1242	34	355
[8,]	55820	1249	36	368
[9,]	7438	1408	39	485
[10,]	919	1641	62	590

Lab 13

Project 4

Credit Card Fraud Detection

Objective:

The objective of this lab is to help students employ machine learning algorithms to detect fraudulent credit cards.

Activity Outcomes:

The activities provide guidance to the following

- Prepare the dataset
- Model the dataset for machine learning algorithms
- Fit the model using logistic regression

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your RStudio

Useful Concepts:

13. //
14. //
15. //
16. //
17. //

27) Solved Lab Activites (Allocated Time 1 Hr. **The time can vary)**

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Loading the required libraries

```
library(ranger)
library(caret)
library(data.table)
library(caTools)
library(pROC)
```

Output:

```
> library(ranger)
> library(caret)
Error: package or namespace load failed for 'caret' in loadNamespace(i, c(lib.loc, .libPaths()), versionCheck = vI[[i]]):
there is no package called 'recipes'
In addition: warning message:
package 'caret' was built under R version 3.6.3
> library(data.table)
> library(caTools)
> library(pROC)
> library(rpart)
> library(rpart.plot)
> library(neuralnet)
> library(gbm)
```

Activity 2:

Downloading and loading the credit card csv

```
creditcard_data <- read.csv("Credit-card-dataset/creditcard.csv")
```

Output:

	Time	V1	V2	V3	V4	V5
1	0	-1.35980713	-0.07278117	2.53634674	1.37815522	-0.338320770
2	0	1.19185711	0.26615071	0.16648011	0.44815408	0.060017649
3	1	-1.35835406	-1.34016307	1.77320934	0.37977959	-0.503198133
4	1	-0.96627171	-0.18522601	1.79299334	-0.86329128	-0.010308880
5	2	-1.15823309	0.87773675	1.54871785	0.40303393	-0.407193377

Activity 3:

Explore the data in the credit card data frame

```
dim(creditcard_data)
head(creditcard_data,6)
tail(creditcard_data,6)
```

Output:

```
> dim(creditcard_data)
[1] 284807      31
> head(creditcard_data, 6)
  Time      V1      V2      V3      V4      V5      V6      V7      V8      V9
1    0 -1.3598071 -0.07278117 2.5363467 1.3781552 -0.33832077 0.46238778 0.23959855 0.09869790 0.3637870
2    0  1.1918571  0.26615071 0.1664801  0.4481541  0.06001765 -0.08236081 -0.07880298 0.08510165 -0.2554251
3    1 -1.3583541 -1.34016307 1.7732093  0.3797796 -0.50319813  1.80049938  0.79146096  0.24767579 -1.5146543
4    1 -0.9662717 -0.18522601 1.7929933 -0.8632913 -0.01030888  1.24720317  0.23760894  0.37743587 -1.3870241
5    2 -1.1582331  0.87773675 1.5487178  0.4030339 -0.40719338  0.09592146  0.59294075 -0.27053268  0.8177393
6    2 -0.4259659  0.96052304 1.1411093 -0.1682521  0.42098688 -0.02972755  0.47620095  0.26031433 -0.5686714
      V10      V11      V12      V13      V14      V15      V16      V17      V18      V19
1  0.09079417 -0.5515995 -0.61780086 -0.9913898 -0.3111694  1.4681770 -0.4704005  0.20797124  0.02579058  0.40399296
2 -0.16697441  1.6127267  1.06523531  0.4890950 -0.1437723  0.6355581  0.4639170 -0.11480466 -0.18336127 -0.14578304
3  0.20764287  0.6245015  0.06608369  0.7172927 -0.1659459  2.3458649 -2.8900832  1.10996938 -0.12135931 -2.26185710
4 -0.05495192 -0.2264873  0.17822823  0.5077569 -0.2879237 -0.6314181 -1.0596472 -0.68409279  1.96577500 -1.23262197
5  0.75307443 -0.8228429  0.53819555  1.3458516 -1.1196698  0.1751211 -0.4514492 -0.23703324 -0.03819479  0.80348692
6 -0.37140720  1.3412620  0.35989384 -0.3580907 -0.1371337  0.5176168  0.4017259 -0.05813282  0.06865315 -0.03319379
```

Activity 4:

Find the summary of the data

```
table(creditcard_data$Class)
summary(creditcard_data$Amount)
names(creditcard_data)
var(creditcard_data$Amount)
sd(creditcard_data$Amount)
```

Output:

```
> table(creditcard_data$Class)
 0      1
284315 492
> summary(creditcard_data$Amount)
   Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
  0.00    5.60    22.00    88.35    77.17 25691.16
> names(creditcard_data)
 [1] "Time"  "V1"    "V2"    "V3"    "V4"    "V5"    "V6"    "V7"    "V8"    "V9"    "V10"   "V11"   "V12"
[14] "V13"   "V14"   "V15"   "V16"   "V17"   "V18"   "V19"   "V20"   "V21"   "V22"   "V23"   "V24"   "V25"
[27] "V26"   "V27"   "V28"   "Amount" "Class"
> var(creditcard_data$Amount)
[1] 62560.07
> sd(creditcard_data$Amount)
[1] 250.1201
```

Activity 5:

Scale the data to remove extreme values

```
head(creditcard_data,6)
creditcard_data$Amount=scale(creditcard_data$Amount)
NewData=creditcard_data[,-c(1)]
head(NewData)
```

Output:

	V1	V2	V3	V4	V5
1	-1.35980713	-0.07278117	2.53634674	1.37815522	-0.338320770
2	1.19185711	0.26615071	0.16648011	0.44815408	0.060017649
3	-1.35835406	-1.34016307	1.77320934	0.37977959	-0.503198133
4	-0.96627171	-0.18522601	1.79299334	-0.86329128	-0.010308880
5	-1.15823309	0.87773675	1.54871785	0.40303393	-0.407193377

Activity 6:

Model the data to train data and test data

```
set.seed(123)
data_sample = sample.split(NewData$Class,SplitRatio=0.80)
train_data = subset(NewData,data_sample==TRUE)
test_data = subset(NewData,data_sample==FALSE)
dim(train_data)
dim(test_data)
```

Output:

```
set.seed(123)
data_sample = sample.split(NewData$Class,SplitRatio=0.80)
train_data = subset(NewData,data_sample==TRUE)
test_data = subset(NewData,data_sample==FALSE)
dim(train_data)
```

```
## [1] 227846    30
```

```
dim(test_data)
```

```
## [1] 56961    30
```

```
Logistic_Model=glm(Class~.,test_data,family=binomial())
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(Logistic_Model)
```

```
##
## Call:
## glm(formula = Class ~ ., family = binomial(), data = test_data)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -4.9019  -0.0254  -0.0156  -0.0078   4.0877
```

Activity 7:

Fit the model using logistic regression, with family as binomial


```
Logistic_Model = glm(Class~., test_data,family = binomial())
summary(Logistic_Model)
plot(Logistic_Model)
lr.predict <- predict(Logistic_Model,train_data, probability = TRUE)
auc.gbm = roc(test_data$Class, lr.predict, plot = TRUE, col = "blue")
```

Output:

```
> summary(Logistic_Model)

Call:
glm(formula = class ~ ., family = binomial(), data = test_data)

Deviance Residuals:
    Min       1Q   Median       3Q      Max 
-4.9019  -0.0254  -0.0156  -0.0078   4.0877 

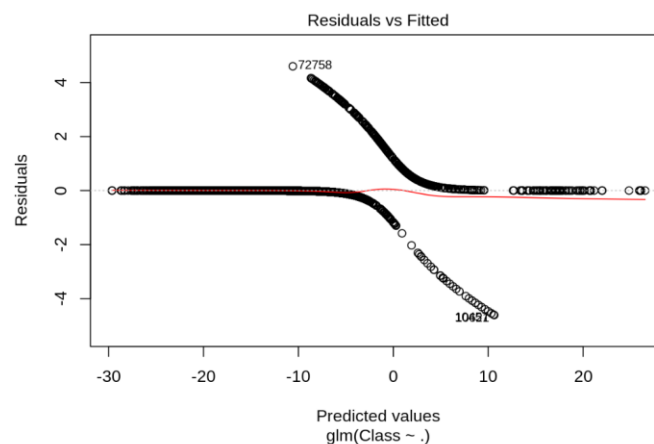
Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -12.52800    10.30537  -1.216   0.2241
V1           -0.17299     1.27381  -0.136   0.8920
V2            1.44512     4.23062   0.342   0.7327
V3            0.17897     0.24058   0.744   0.4569
V4            3.13593     7.17768   0.437   0.6622
V5            1.49014     3.80369   0.392   0.6952
V6           -0.12428     0.22202  -0.560   0.5756
V7            1.40903     4.22644   0.333   0.7388
V8           -0.35254     0.17462  -2.019   0.0435 *
V9            3.02176     8.67262   0.348   0.7275
V10          -2.89571     6.62383  -0.437   0.6620

---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1443.40  on 56960  degrees of freedom
Residual deviance:  378.59  on 56931  degrees of freedom
AIC: 438.59

Number of Fisher Scoring iterations: 17
```

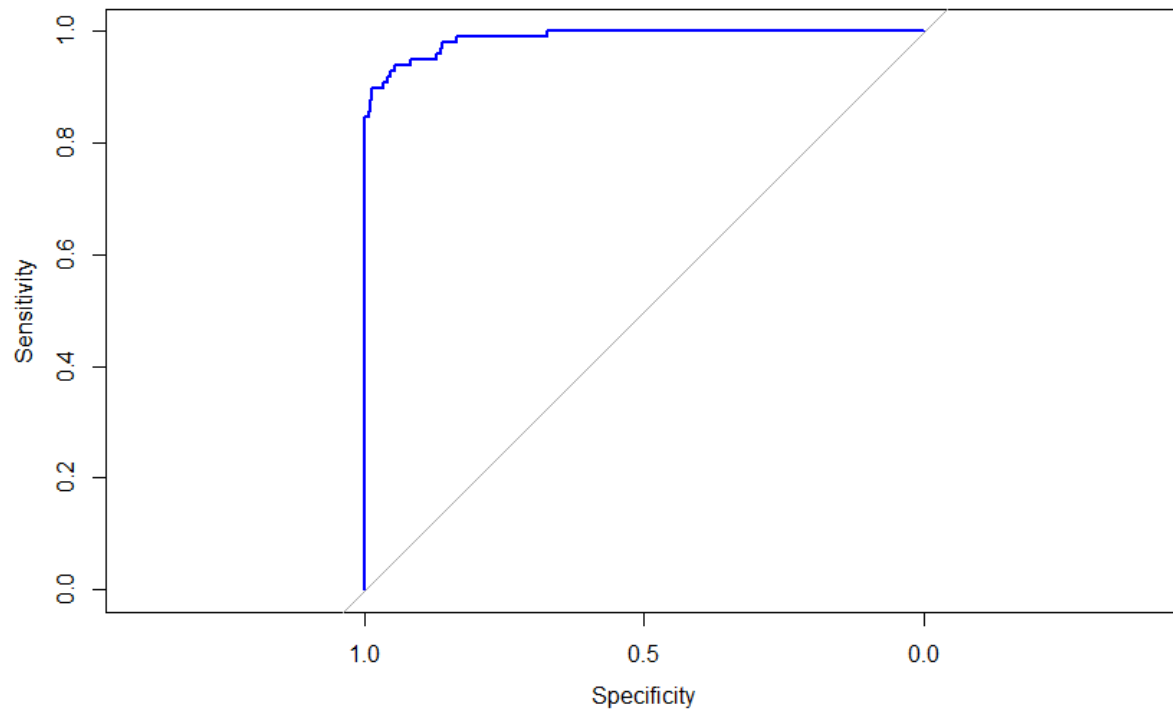


Activity 8:

Assess the performance of the model

```
lr.predict <- predict(Logistic_Model, test_data, probability =TRUE)
auc_gbm = roc(test_data$Class, lr_predict, plot= TRUE, col = "blue")
```

Output:



28) Graded Lab Tasks((Allocated Time 1 Hr. **The time can vary)**
(The instructor can design the complexity and frequency of unsolved lab tasks as per requirement of the lab. The lab tasks assigned by the instructor should be evaluated in the same lab.)

Task 01:

Implement the logistic regression using the following families:

1. Gaussian
2. Poisson

What are the differences are observed? Which family is the best?

Task 1 Activity 1:

Loading the required libraries

```
library(ranger)
library(caret)
library(data.table)
library(caTools)
library(pROC)
library(rpart)
library(rpart.plot)
library(neuralnet)
library(gbm)
```

Output:

```
> library(ranger)
> library(caret)
Error: package or namespace load failed for 'caret' in loadNamespace(i, c(lib.loc, .libPaths()), versioncheck = vi[[i]]):
  there is no package called 'recipes'
In addition: warning message:
package 'caret' was built under R version 3.6.3
> library(data.table)
> library(caTools)
> library(pROC)
> library(rpart)
> library(rpart.plot)
> library(neuralnet)
> library(gbm)
```

Task 1 Activity 2:

Downloading and loading the credit card csv

```
creditcard_data <- read.csv("Credit-card-dataset/creditcard.csv")
```

Output:

	Time	V1	V2	V3	V4	V5
1	0	-1.35980713	-0.07278117	2.53634674	1.37815522	-0.338320770
2	0	1.19185711	0.26615071	0.16648011	0.44815408	0.060017649
3	1	-1.35835406	-1.34016307	1.77320934	0.37977959	-0.503198133
4	1	-0.96627171	-0.18522601	1.79299334	-0.86329128	-0.010308880
5	2	-1.15823309	0.87773675	1.54871785	0.40303393	-0.407193377

Task 1 Activity 3:

Scale the data to remove extreme values

```
head(creditcard_data,6)
creditcard_data$Amount=scale(creditcard_data$Amount)
NewData=creditcard_data[,-c(1)]
head(NewData)
```

Output:

	V1	V2	V3	V4	V5
1	-1.35980713	-0.07278117	2.53634674	1.37815522	-0.338320770
2	1.19185711	0.26615071	0.16648011	0.44815408	0.060017649
3	-1.35835406	-1.34016307	1.77320934	0.37977959	-0.503198133
4	-0.96627171	-0.18522601	1.79299334	-0.86329128	-0.010308880
5	-1.15823309	0.87773675	1.54871785	0.40303393	-0.407193377

Task 1 Activity 4:

Model the data to train data and test data

```
set.seed(123)
data_sample = sample.split(NewData$Class,SplitRatio=0.80)
train_data = subset(NewData,data_sample==TRUE)
test_data = subset(NewData,data_sample==FALSE)
dim(train_data)
dim(test_data)
```

Output:

```
> set.seed(123)
> data_sample = sample.split(NewData$Class, SplitRatio = 0.80)
> train_data = subset(NewData, data_sample == TRUE)
> test_data = subset(NewData, data_sample == FALSE)
> dim(train_data)
[1] 227846    30
> dim(test_data)
[1] 56961    30
```

Task 1 Activity 7:

Fit the model using logistic regression, with family as Gaussian

```
Logistic_Model = glm(Class~., test_data,family = gaussian ())
summary(Logistic_Model)
plot(Logistic_Model)
lr.predict <- predict(Logistic_Model,train_data, probability = TRUE)
auc.gbm = roc(test_data$Class, lr.predict, plot = TRUE, col = "blue")
```

Output:

```
> summary(Logistic_Model)
```

Call:
glm(formula = class ~ ., family = binomial(), data = test_data)

Deviance Residuals:

	Min	1Q	Median	3Q	Max
	-4.9019	-0.0254	-0.0156	-0.0078	4.0877

Coefficients:

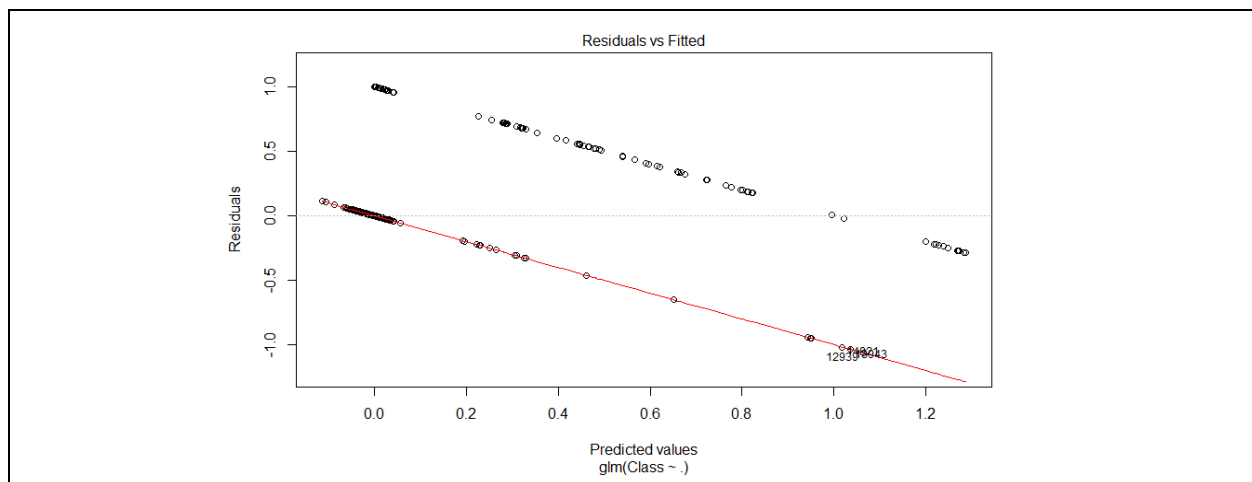
	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-12.52800	10.30537	-1.216	0.2241
V1	-0.17299	1.27381	-0.136	0.8920
V2	1.44512	4.23062	0.342	0.7327
V3	0.17897	0.24058	0.744	0.4569
V4	3.13593	7.17768	0.437	0.6622
V5	1.49014	3.80369	0.392	0.6952
V6	-0.12428	0.22202	-0.560	0.5756
V7	1.40903	4.22644	0.333	0.7388
V8	-0.35254	0.17462	-2.019	0.0435 *
V9	3.02176	8.67262	0.348	0.7275
V10	-2.89571	6.62383	-0.437	0.6620

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.0008706217)

Null deviance: 97.831 on 56960 degrees of freedom
Residual deviance: 49.565 on 56931 degrees of freedom
AIC: -239684

Number of Fisher Scoring iterations: 2

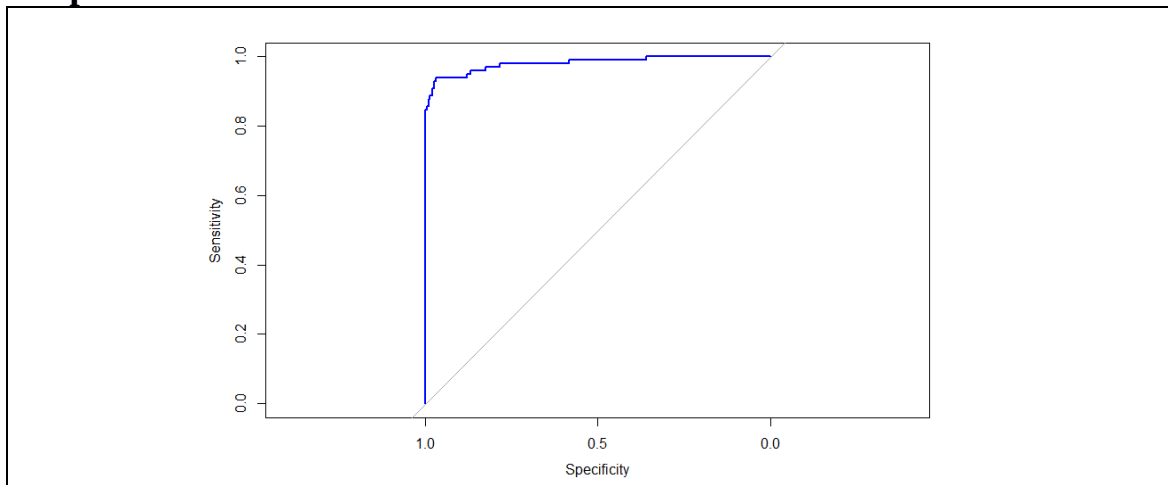


Task 1 Activity 8:

Assess the performance of the model

```
lr_predict <- predict(Logistic_Model, test_data, probability = TRUE)
auc_gbm = roc(test_data$Class, lr_predict, plot = TRUE, col = "blue")
```

Output:



Task 1 Activity 9:

Fit the model using logistic regression, with family as Poisson

```
Logistic_Model = glm(Class~., test_data,family = poisson ())
summary(Logistic_Model)
plot(Logistic_Model)
lr.predict <- predict(Logistic_Model,train_data, probability = TRUE)
auc.gbm = roc(test_data$Class, lr.predict, plot = TRUE, col = "blue")
```

Output:

```
Call:
glm(formula = Class ~ ., family = poisson(), data = test_data)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.4246	-0.0448	-0.0270	-0.0115	3.8348

Coefficients:

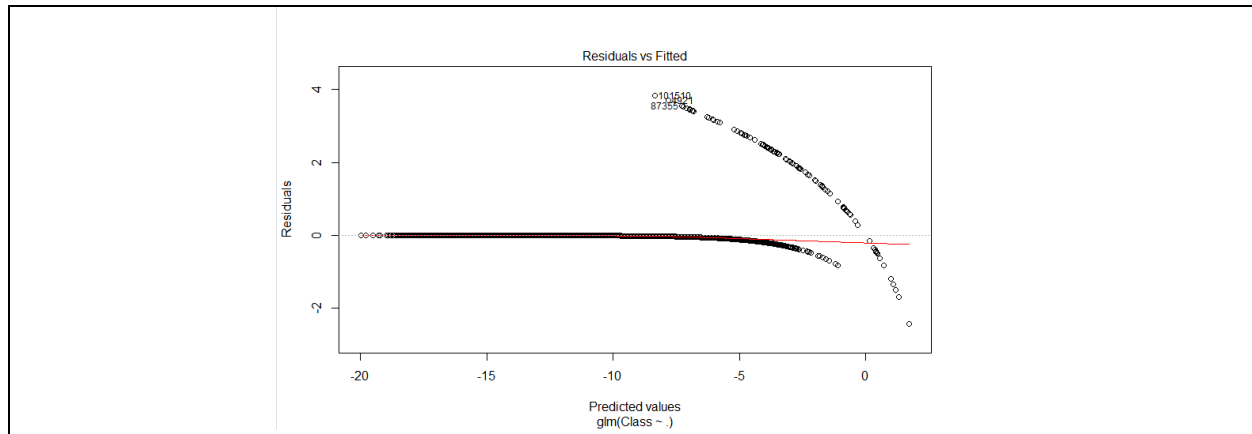
	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-8.784947	1.605005	-5.473	4.41e-08 ***
v1	-0.151496	0.202292	-0.749	0.4539
v2	0.508961	0.661975	0.769	0.4420
v3	0.043368	0.087806	0.494	0.6214
v4	1.123190	1.109501	1.012	0.3114
v5	0.305847	0.601457	0.509	0.6111
v6	-0.284933	0.161952	-1.759	0.0785 .
v7	0.477644	0.655680	0.728	0.4663
v8	-0.151968	0.059776	-2.542	0.0110 *
v9	0.948355	1.340368	0.708	0.4792
v10	-0.810760	1.027622	-0.789	0.4301

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 1247.57 on 56960 degrees of freedom
Residual deviance: 562.33 on 56931 degrees of freedom
AIC: 818.33

Number of Fisher Scoring iterations: 15

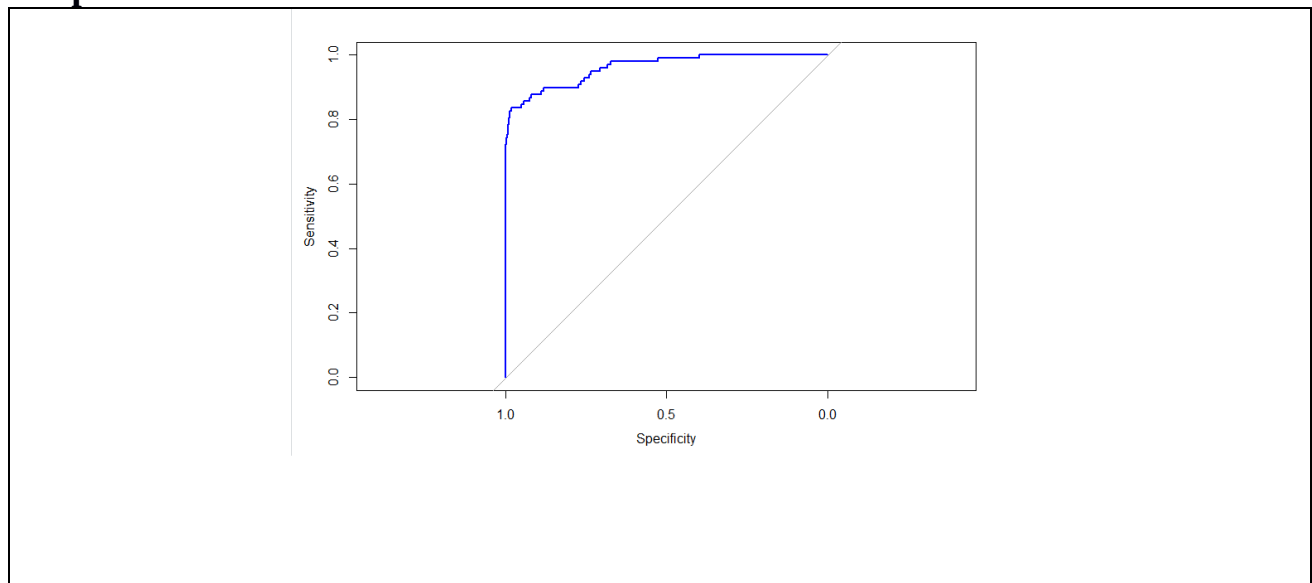


Task 1 Activity 10:

Assess the performance of the model

```
lr.predict <- predict(Logistic_Model, test_data, probability = TRUE)
auc_gbm = roc(test_data$Class, lr_predict, plot = TRUE, col = "blue")
```

Output:



Answer:

Different graphs, explanation of what each Predicted Values – Residuals graphs tells. Different Fisher Scoring Iterations and their meaning (lower fisher scoring means lower iterations it took to fit the model). The obvious choice of family is binomial as there are only two outcomes for the response variable i.e., fraudulent, or non fraudulent transaction.

Lab 14

Credit Card Fraud Detection

Objective:

The objective of this lab is to help students employ machine learning algorithms to detect fraudulent credit cards.

Activity Outcomes:

The activities provide guidance to the following

- Use other machine learning models
- Plot the performance curves for each method

Instructor Note:

You need to have a working internet, along with administrative rights on the Laptop/PC on which you are setting up your RStudio

Useful Concepts:

1. //
2. //
3. //
4. //
5. //

29) Solved Lab Activites (Allocated Time 1 Hr. **The time can vary**)

(The instructor can design the complexity and frequency of the solved lab activities as per requirement of the lab.)

Activity 1:

Loading the required libraries

```
library(rpart)
library(rpart.plot)
library(neuralnet)
library(gbm)
```

Output:

Activity 2:

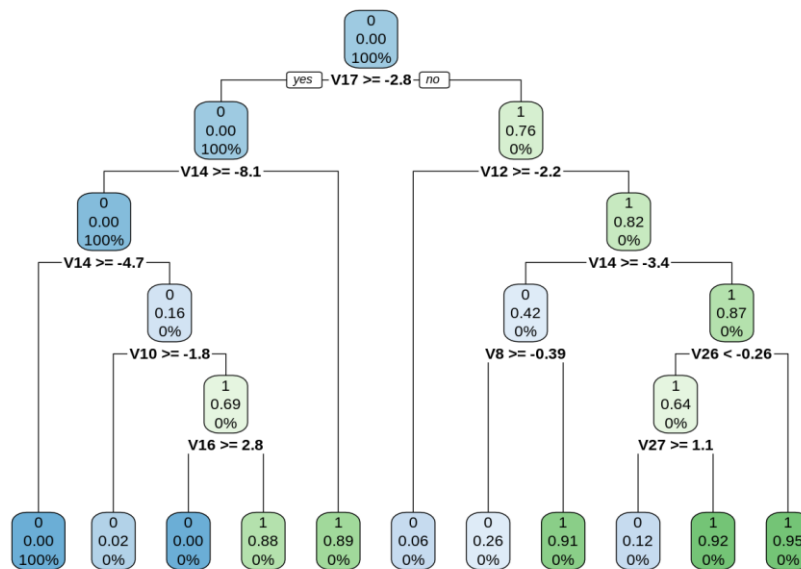
Plotting the decision tree


```

decisionTree_model <- rpart(Class ~ . , creditcard_data, method = 'class')
predicted_val <- predict(decisionTree_model, creditcard_data, type = 'class')
probability <- predict(decisionTree_model, creditcard_data, type = 'prob')
rpart.plot(decisionTree_model)

```

Output:



Activity 3:

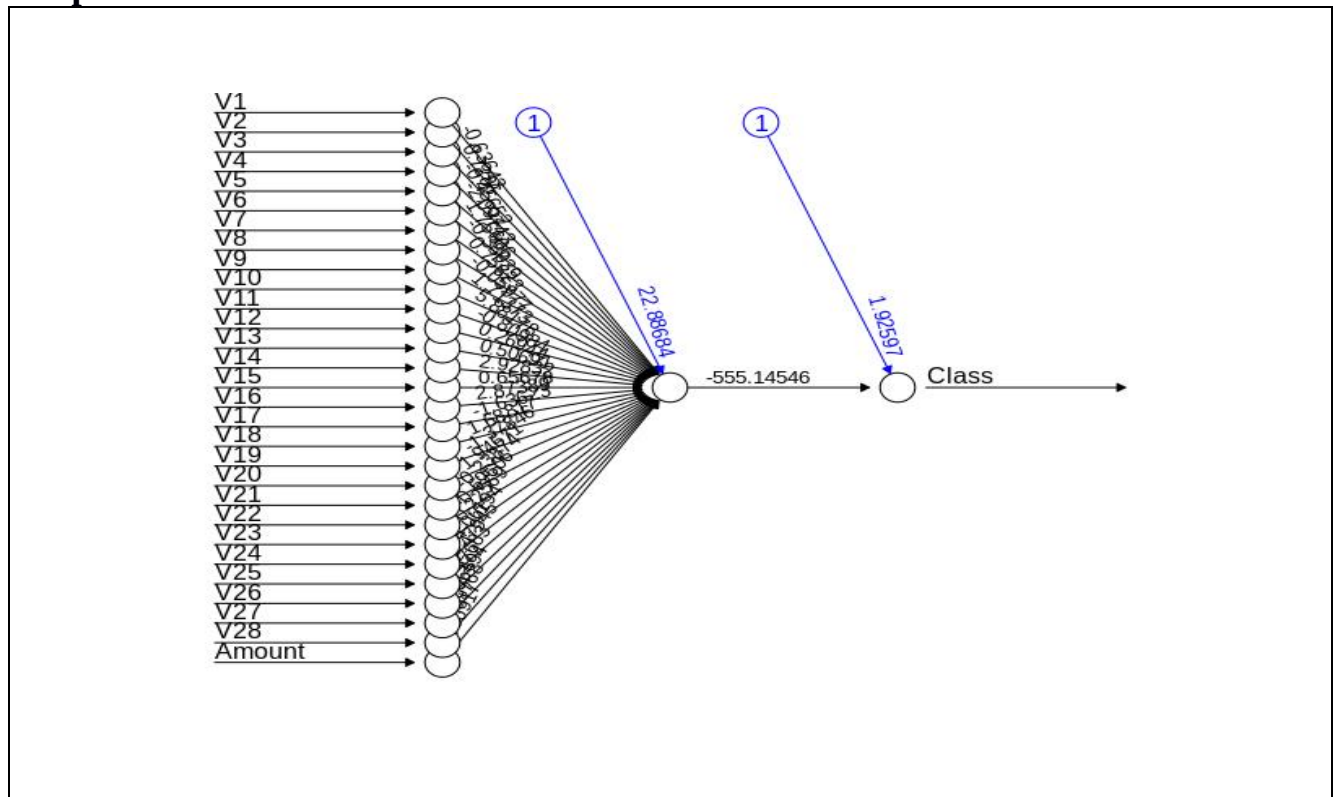
Plotting ANN model

```

ANN_model =neuralnet (Class~.,train_data,linear.output=FALSE)
plot(ANN_model)
predANN=compute(ANN_model,test_data)
resultANN=predANN$net.result
resultANN=ifelse(resultANN>0.5,1,0

```

Output:



30) Graded Lab Tasks((Allocated Time 1 Hr. **The time can vary)**
(The instructor can design the complexity and frequency of unsolved lab tasks
as per requirement of the lab. The lab tasks assigned by the instructor should
be evaluated in the same lab.)

Task 01:

1. *Plot a Grading Boosting Model*
2. *Analyse the results of the plot*
3. *What do the plots of different algorithms tell*

Task 1 Activity 1:

Plotting Grading Boosting Model

```
system.time(  
  model_gbm <- gbm(Class ~ .  
    , distribution = "bernoulli"  
    , data = rbind(train_data, test_data)  
    , n.trees = 500  
    , interaction.depth = 3  
    , n.minobsinnode = 100  
    , shrinkage = 0.01  
    , bag.fraction = 0.5  
    , train.fraction = nrow(train_data) / (nrow(train_data) + nrow(test_data))  
  )  
  )  
gbm.iter = gbm.perf(model_gbm, method = "test")  
model.influence = relative.influence(model_gbm, n.trees = gbm.iter, sort. = TRUE)  
plot(model_gbm)
```

Output:

